Smooth Emulator & Training Point Optimizer

User Manual

A BAND Collaboration Project

https://bandframework.github.io

Scott Pratt, Oleh Savchuk, Eren Erdogan, Ekaksh Kataria

Department of Physics and Facility for Rare Isotope Beams

Michigan State University, East Lansing Michigan, 48824

March 31, 2026

MICHIGAN STATE UNIVERSITY

This project was supported by the National Science Foundation through the Office of Cyberinfrastructure for Sustained Scientific Motivation (CSSI)

Contents

1 Overview

Smooth Emulator software is designed with the idea that the full-model to be emulated is *smooth*. I.e., if one were represent the behavior as a Taylor expansion, one would expect higher-order terms to contribute less than lower-order terms. The programs that choose the training points are based on such an assumption, and the emulator's mathematical form also presumes such behavior. Applying the software to emulation of a full-model that has sharp peaks or valleys is probably dubious. The programs tend to run quickly as long as the number of model-parameters is of the order a dozen or less.

The software performs three basic functions.

I. Provide an optimum set of training points in model parameter space, at which full model runs will performed to then tune the emulator. The User must first provide a list of model parameters and a description of their priors, along with a measure of the relative importance of each model parameter.

II. After the User provides a list of observables, the User runs their full model at each of the training points and provides values and uncertainties for each observable at each training point. *Smooth Emulator* software then creates and tunes emulators for each observable. The emulator form is based on two hyper-parameters, an amplitude and a convergence length. Both hyper-parameters are found by the tuning procedure. The emulator is mathematically equivalent to a Gaussian process (GP) emulator with a quadratic kernel, with the convergence length being the correlation length of a GP emulator.

III. Once the User provides a list of experimentally measured observables and uncertainties, an MCMC program samples the prior and provides a sampling of points in the model-parameter space that are consistent with the posterior likelihood. An analysis program uses the list of representative points to calculate the mean and covariances of the posterior distribution. The program also provides the resolving power of how each observable constrains each model parameter.

The structure is designed so that the User stores all the information for a specific analysis in a specific analysis directory, e.g. `${MY_ANALYSIS}/`. All programs, aside from Python scripts for plotting, are then run from that directory. A subdirectory, `${MY_ANALYSIS}/smooth_data/Info`, contains simple text files edited by the User to describe the model-parameter priors, the observables to be emulated and evaluated, and the experimentally observed values. A second subdirectory, `${MY_ANALYSIS}/smooth_data/Options` contains text files defining options the User can set related to the *Smooth Emulator* software. The training point optimization software creates directories `${MY_ANALYSIS}/smooth_data/FullModelRuns/runI` where each training point, `.../runI`, refers to a single point in model-parameter space used for training. The training point optimizer program creates a listing of a set of training points, where the values of the model parameters for each training point are written to files. The User's job is then to create text files giving the values of the observables calculated by the full model for each training point. These files are stored in the directory `${MY_ANALYSIS}/smooth_data/FullModelRuns/`. The MCMC program writes the MCMC trace information to files in `smooth_data/MCMC/`. *Smooth Emulator* provides programs that analyze the trace to provide averages and covariances of the the posterior, as well as measures of the resolving power. Several Python scripts are located in `${MY_ANALYSIS}/figs/`.

Smooth Emulator software is designed to be modular. Programs for each of the three functionalities listed above can be performed independently because files are all in simple text formats. For example, one can generate the training points, then use those training points to train emulators from other software packages. However, the User will likely have to invest time translating text files to formats used by other programs. Aside from the plotting scripts, `Smooth Emulator` software is written in C++. The software, and this manual, were built on the assumption that the User has some familiarity with C++. Defining options, setting priors, are all done by editing simple text files. The software then performs its task by reading such files. Given that generating the data from the full-model runs used to train the emulator is likely to be a lengthy process, perhaps involving weeks of running on large facilities, the software is designed with the idea that a User will wish to have data written in simple formats so that it can be readily copied as need be.

2 Installation and Getting Started

2.1 Overview

This section describes how to download and install *Smooth Emulator* software. The software is downloaded as part of the BAND git repository. *Smooth Emulator* software is all contained within a single directory of that repository. Once downloaded, the *Smooth Emulator* portion of the software can be copied to a location of the User's choosing. The source code in that directory can then be compiled. For performing analyses, the User should copy a directory from the distribution, which then serves as a template. A separate directory should be copied for any given analysis. The source code is split into different directories. One set of codes is used to produce libraries. Other codes are the main programs, which tend to be simple short programs, which then make calls to the libraries. These short codes are designed so that they can be copied and altered to address the User's needs. The template analysis directory also serves for the tutorial described in Chapter 10.

The software is designed to be run interactively, from the command line in standard Unix-like shells. The commands are designed to work when invoked from an analysis directory, like those from the template, which have the required initialization and options files in standard locations. The User can also run the software in the batch mode, though that is usually not necessary given how quickly the operations typically proceed.

2.2 Prerequisites

Smooth Emulator programs should run on Mac OS or Linux, but is not supported for Windows OS. The software is largely written in C++, though there are some Python scripts for plotting results. In addition to a C++ compiler (CMake files assume you can accommodate the C++ 20 standard), the user needs the following software installed.

- git
- CMake (version 4.x)
- Eigen3 (Linear Algebra Package)
- *Python/Matplotlib (for generating plots)
- *pybind11 (To call emulator from within Python script)
- *PDF viewer

(* not necessary)

CMake is an open-source, cross-platform build system that helps automate the process of compiling and linking software projects. Hopefully, CMake will perform the needed gymnastics to find the Eigen3 installation. To install CMake, either visit the CMake website (https://cmake.org/), or use the system's package manager for the specific system. For example, on Mac OS, if one uses *homebrew* as a package manager, the command is

```
% brew install cmake
```

Eigen is a C++ template library for vector and matrix operations, i.e. linear algebra. The user can visit the Eigen website (https://eigen.tuxfamily.org/dox/), or use their system's package manager. For example on Mac OS with *homebrew*,

```
% brew install eigen
```

The provided plotting scripts are written in Python using Matplotlib. Most systems typically include Python, but Matplotlib often requires installation. On some systems Matplotlib can be added via the `pip` Python package manager. In other cases it can be added via the system package manager directly. Some Users may wish to call *Smooth Emulator* functionality from within a Python script, most likely if they have their own MCMC software, but want to call a function that returns a prediction and uncertainty for the observables as a function of the model parameters. The foundation for doing this is provided in the release, but requires installing pybind11. This can also be installed via `pip`. It is crucial that pybind11 be installed consistently with the version of Python being used. On a Mac, one can consistently install all the required packages from the system package manager, *homebrew*.

```
% brew install python
% brew install numpy
% brew install python-matplotlib
% brew install pybind11
```

Because pybind11 installation is often fraught with compatibility problems, it is not recommended to bother with its installation unless the User wishes to call the emulator from within a Python script or from a Python notebook.

A PDF viewer will be necessary to view the plots generated by the Python scripts above. The software-testing script calls the PDF viewers to compare figures with previously generated ones. On the Mac, it calls Preview. For Linux it looks for either Okular or Evince. If those are not installed the User can simply view the generated PDF files later using their own viewer.

2.3 Downloading the Repository and Creating Working Directories

The software requires downloading the BAND framework software repository into some directory. Should that be in the User's home directory, the User might enter

```
/Users/CarlosSmith% git clone https://github.com/bandframework/bandframework.git
```

Within the repository, there will be a directory `/Users/CarlosSmith/bandframework/software/SmoothEmulator/`. All the relevant functionality of *Smooth Emulator* is contained within this directory. The User can either leave this directory in place, or make a copy to a new location. If the User plans on making changes on the underlying software and rebuilding the core libraries, it would make sense to copy this directory to a new location. Otherwise, the User might wish to simply use the repository version, but a `git pull` command might create conflicts. Throughout the manual the phrase `${SMOOTH_HOME}` will refer to this directory. The software and CMake files for creating the *Smooth Emulator* libraries is stored in the `${SMOOTH_HOME}/software/` directory.

If the User downloaded the `bandframework` repository into their home directory, they might copy the *Smooth Emulator* software as follows.

```
/Users/CarlosSmith% cp -r bandframework/software/SmoothEmulator ./SmoothEmulator
```

The User should then define an environmental variable to refer to this location.

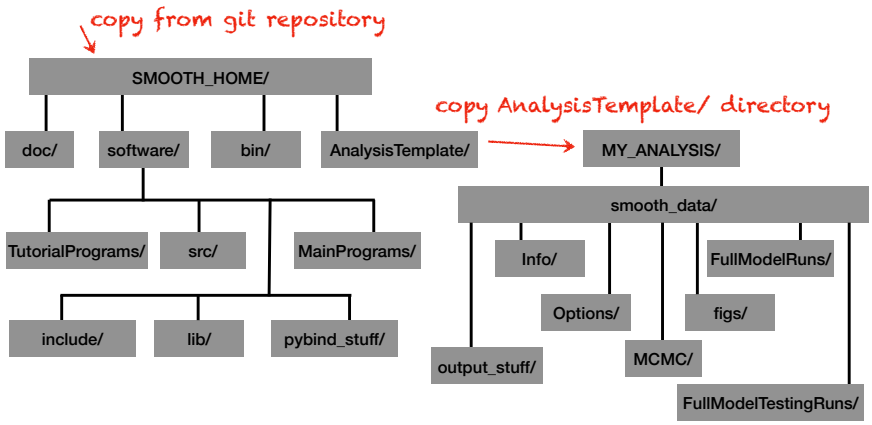


Figure 2.1: Directory Structure: The User clones the repository and copies the *Smooth Emulator* segment into some location, which will be referred to as `${SMOOTH_HOME}`. The User copies the `AnalysisTemplate` directory to `${MY_ANALYSIS}`. The User will likely make multiple copies of the analysis directory for different projects or for different analyses with different options. The main source code and compiled libraries will be built and kept within the `${SMOOTH_HOME}/` tree. Executables are stored in `${SMOOTH_HOME}/bin/`. The software is designed to be run from the command line in the `${MY_ANALYSIS}` directory. Data used and created by *Smooth Emulator* software is stored in `${MY_ANALYSIS}/smooth_data/`.

```
% export SMOOTH_HOME=/Users/CarlosSmith/SmoothEmulator
```

If the `../bandframework/software/SmoothEmulator` directory was moved and renamed to something else, then that location should replace `/Users/CarlosSmith/SmoothEmulator` above. This location gets referenced in some of the *Smooth Emulator* scripts, so it is recommended to copy this definition into one's `.zshrc` or `.bashrc` script so that one needn't redefine it. It is also recommended that the User redefine their path. For convenience, this statement should also be in the User's `.zshrc` or `.bashrc` file.

```
% export PATH=$PATH:${SMOOTH_HOME}/bin
```

The User should create a directory from which the User would perform an analysis. This is most easily accomplished by copying the analysis template directory from the *Smooth* distribution. For example

```
% cp -r ${SMOOTH_HOME}/AnalysisTemplate /Users/CarlosSmith/MY_ANALYSIS
```

The User may choose any other name besides `MY_ANALYSIS`. Hence forth, `${MY_ANALYSIS}` is a name chosen by the User, and will refer to this directory, including the path, from which the User will perform most of the analysis. Thus, the User may wish to have several such directories, located according to the User's preference. Any time a new analysis is performed with new parameters, and if the User wishes to save the previous analysis, a new `${MY_ANALYSIS}/` directory should be created. The software is designed to be run from the command line from the `${MY_ANALYSIS}/` directory. All data related to the specific analysis will be stored in the `${MY_ANALYSIS}/smooth_data/` directory.

The directory structure is shown in Fig. 2.1. As stated above, for the remainder of this manual, `${SMOOTH_HOME}/` and `${MY_ANALYSIS}/` will be used to denote the location of these directories.

2.4 Compiling the Software

First, change into software directories, create the makefiles with CMake, then compile them.

```
% cd ${SMOOTH_HOME}/software
${SMOOTH_HOME}/software% cmake .
${SMOOTH_HOME}/software% make
```

Executables should now appear in `${SMOOTH_HOME}/bin/` and libraries are created in `${SMOOTH_HOME}/software/lib/` If the User plans on using the `pybind11` functionality. They should perform an additional compilation.

```
% cd ${SMOOTH_HOME}/software
${SMOOTH_HOME}/software% cd pybind_stuff
${SMOOTH_HOME}/software/pybind_stuff% cmake .
${SMOOTH_HOME}/software/pybind_stuff% make
```

There seems to be a common problem that CMake misreports the path of the `Eigen` installation. If the User should get an error stating that the Eigen header files cannot be found, the User can set an environmental variable to point to the correct location, e.g.,

```
% export EIGEN3_INCLUDE_DIR=/opt/homebrew/include/eigen3/
```

The final arguments may need to be changed depending on the User's location of the packages. If the User wishes to choose a specific C++ compiler, the `cmake` command should be replaced with `cmake -D CMAKE_CXX_COMPILER=g++`, or whichever compiler is preferred. On the Mac, the pybind11 software seemed to work better with the g++ compiler than with the clang compiler.

2.5 The `${MY_ANALYSIS}/smooth_data` Directory

Within `${MY_ANALYSIS}/smooth_data/` there are several sub-directories (assuming it was created from the template). The first is `smooth_data/Info/` which contains three text files. Information about the model parameters, and their priors is stored in `smooth_data/Info/prior_info.txt`, and information about the observables is stored in `smooth_data/Info/observable_info.txt`. The file `smooth_data/Info/experimental_info.txt` stores information about the measurements for each observable, and is not needed for building the emulator, but is used by the MCMC investigation of the posterior.

The `smooth_data/FullModelRuns` data has subdirectories `run0/`, `run1/` $\cdots$. Each subdirectory stores information describing a full model run. The file `smooth_data/FullModelRuns/runX/model_parameters.txt` stores the model parameters used for run "`X`". This file can be provided by the User, or it can be created by the training-point optimization codes. Once those files are created, it is the User's responsibility to run the full model and record the observables in `smooth_data/FullModelRuns/runX/obs.txt`. Given those observables, the User can then run the emulator software. If the User runs the MCMC software, the MCMC traces are stored as text files in `smooth_data/MCMC_Trace/`. Examples of Python scripts for graphing using Matplotlib can be found in `smooth_data/figs/`.

The directory `smooth_data/Options` stores three text files, which also must be edited by the User. These files set options for training-point optimization, emulation and MCMC searches respectively.

The smooth_data/ directory has several other subdirectories are, but those are generated by the software, and might or might not, be of interest to the User.

If the User wishes to perform a different analysis, perhaps using a different set of observables, it is recommended to copy the MY_ANALYSIS directory and perform the new analysis in the new location, so that the previous analysis data is not over-written.

2.6 Setting up "Info" files

The smooth_data/Info/ directory contains three files which the User must create and edit before running any of the *Smooth Emulator* software. Each of these files is a simple ascii file. The User may add comments to any of the "Info" files by placing the # symbol at the beginning of the line. Only the first file, smooth_data/Info/prior_info.txt is required for the training-point optimization software. That file and smooth_data/Info/observable_info.txt are both necessary for emulation. All three files, including smooth_data/Info/experimental_info.txt, which lists the experimental measurements of the observables used to constrain the model-parameter space, are required for running the MCMC software.

2.6.1 smooth_data/Info/prior_info.txt

This file defines the model parameters and their priors. The format of the simple ascii file is:

```
parameter_name_1  prior_type_1 xmin_1/centroid_1  xmax_1/Rgauss_1
↪  sensivity_scale_1
parameter_name_2  prior_type_2 xmin_2/centroid_2  xmin_2/Rgauss_2
↪  sensivity_scale_2
   .
parameter_name_N  prior_type_N xmin_N/centroid_N  xmin_N/Rgauss_N
↪  sensivity_scale_N
```

For each of the N model parameters the file devotes one line. The first string (no spaces) is the parameter name. The second string describes the prior type, and must be either gaussian or uniform. The next two entries describe the range of the prior. For uniform priors, the numbers represent the boundaries of the distribution. For a gaussian prior, the first entry represents the centroid of the gaussian and the next entry represents the gaussian's width. The last entry is the sensitivity scale. If all the model parameters are expected to contribute similarly, all the values should be set to unity. However, if there are some parameters for which one expects to contribute in a more linear fashion (higher orders are less important) one can enter a smaller number. None of the sensitivity scales should exceed unity. For less impactful model parameters, where the User would be satisfied with a nearly linear fit, one enters a number below unity. In practice, this is the same as assuming that the convergence parameter is larger than those with sensitivity scales set to unity, the convergence parameters scales inversely with the sensitivity scale. For example, if the convergence parameter, Λ, is set to 2.5, setting the sensitivity scale to 0.5 for some model parameter is equivalent to assuming that the effective value of Λ for that model parameter is 5.0.

2.6.2 smooth_data/Info/observable_info.txt

This file describes the observables. It provides only the n observable names and the point-by-point uncertainties.

```
observable_name_1   ALPHA_1
observable_name_2   ALPHA_1
   .
observable_name_n   ALPHA_N
```

The measurement uncertainties and the uncertainties due to theoretical systematic error are not provided here – they are provided in `smooth_data/Info/experimental_info.txt`, as they do not come into play until one is comparing the model to measurement or observation. The parameter `ALPHA_i` describes only the point-by-point uncertainty. This class of uncertainty arises when the full model has some sort of noisy contribution, i.e. one where if the model were re-run with identical model parameters the observable values would vary. Examples of such models would be observables from simulations of nuclear collisions using a finite number of events. If the observable is expected to vary by some representative value σ_A throughout the prior, `ALPHA_i` is the fraction of σ_A describing the additional variation due to noise. For example if one believed that a mean transverse momentum measurement from a high-energy collider measurement might vary by 100 MeV throughout the prior, and if the noisy contribution was about 1 MeV, then one would set `ALPHA_i` to 0.01. If the noise comes from $N_{\rm sample}$ measurements, then one would probably simply set ALPHA_i to $1/\sqrt{N_{\rm sample}}$. This parameter ensures that should the User choose two training points that are nearly adjacent, the emulator will not attempt to exactly reproduce both points, which would lead to wildly varying, non-smooth, behavior.

2.6.3 smooth_data/Info/experimental_info.txt

The third file describes the measurement and their uncertainties. It is only used when performing the MCMC fit, which occurs after the emulator is trained. The format is

```
observable_name_1   measurement_1  exp_uncertainty_1   theory_uncertainty_1
observable_name_2   measurement_2  exp_uncertainty_2   theory_uncertainty_2
   .
observable_name_N   measurement_N  exp_uncertainty_N   theory_uncertainty_N
```

The observable names must be identical to, or a subset of, those from `smooth_data/Info/observable_info.txt`. The measurements and their uncertainties are typically taken from the experimental publication. The last entry in each line is the systematical theoretical uncertainty, i.e. that arising from the physics missing from the model. For example, if the model is missing a certain bit of physics, or if that physics is described through an imperfect approximation, then one would expect the model calculations to vary from the experimental values even if the model parameters were perfectly chosen. In practice, the two uncertainties contribute as a single uncertainty, where the two are added in quadrature.

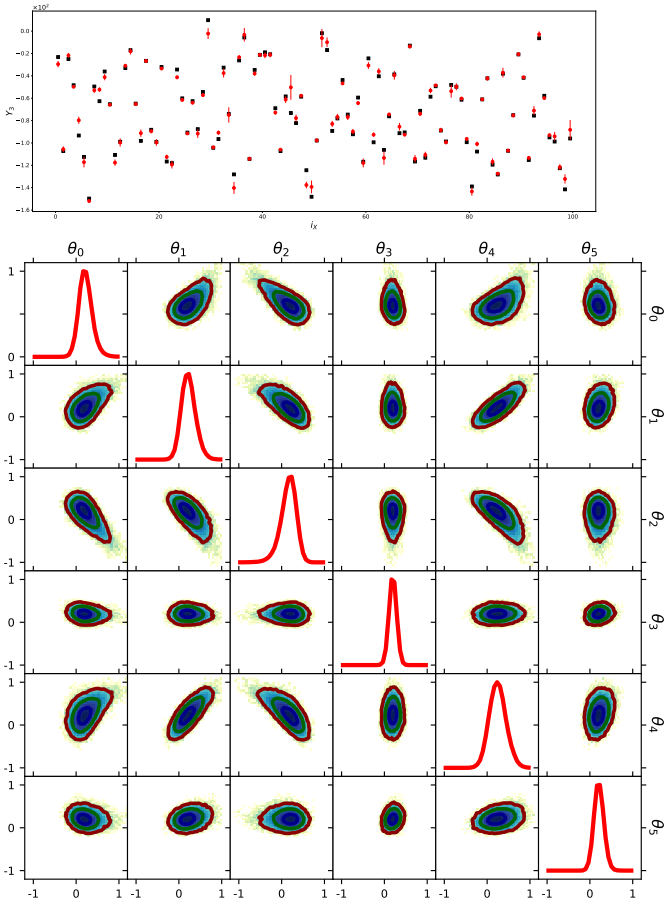
Figure 2.2: Figures created by the `software_test.sh` script. These same figures will be created by completing the tutorial in Chapter 10

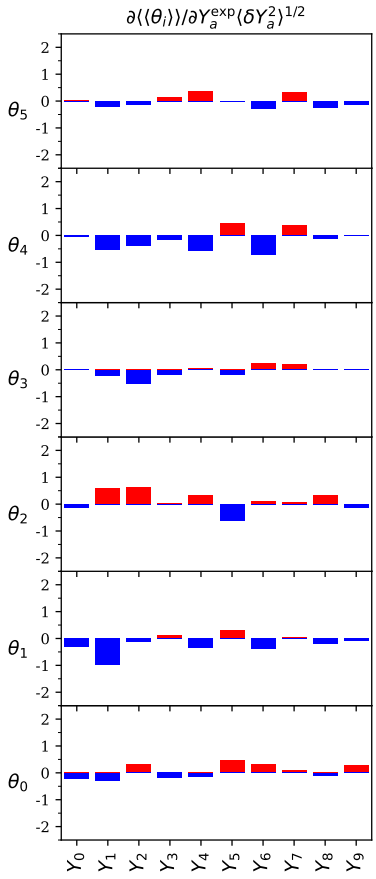
2.7 Testing the Installation

The User can go to the tutorial analysis directory created by copying the template. There should be a script there, `${MY_ANALYSIS}/software_test.sh`. This script runs all the commands used in the tutorial. This includes generating figures. The script calls a PDF viewer to open pre-made versions of the plots and compare the newly generated plots to the pre-made versions.

```
${MY_ANALYSIS}% smooth_test.sh
.
.
```

If all goes well, three pairs of matching figures should ultimately appear.





To test the `pybind11` functionality, enter the command below. The output should be:

```
${MY_ANALYSIS}% pybind_test.sh
.
.
Adding this to path: XXX/software/lib
X= [20. 20. 20. 20. 20. 20.]
Y[ 0 ]= -87.52874076690921  SigmaY= 1.581866454534034
```

```
Y[ 1 ]= -11.271353985726321  SigmaY= 0.5174129111757715
Y[ 2 ]= 142.7366961537101  SigmaY= 0.224412945072997
Y[ 3 ]= -75.69108719689788  SigmaY= 0.37636695243649665
Y[ 4 ]= 77.5122131352153  SigmaY= 0.9065709798513465
Y[ 5 ]= 58.88131815229454  SigmaY= 1.2925008270736835
Y[ 6 ]= 134.41231285717063  SigmaY= 1.3337258303822137
Y[ 7 ]= 137.45383706357387  SigmaY= 0.5634991046716276
Y[ 8 ]= 5.924468079091237  SigmaY= 0.5474124140829644
Y[ 9 ]= 106.78915673976991  SigmaY= 0.553889676442671
```

This python script called the emulator and calculated each of ten observables, $Y_0 ... Y_9$ at the point in model-parameter space, $X = (20, 20, 20, 20, 20, 20)$. It then printed the emulated value and the uncertainty for each of 10 observables for that point.

3 Underlying Theory of *Smooth Emulator*

3.1 Overview

A detailed explanation of the underlying theory used by *Smooth Emulator* software for emulation and for training-point optimization is provided in the accompanying paper `smoothdraft.pdf`, which can be found in the same directory as this paper.

The choice of model emulators, $E(\vec{\theta})$, depends on the prior understanding of the model being emulated, $M(\vec{\theta})$, which we refer to as the "full model". If one knows that a function is linear, then a linear fit is clearly the best choice. Whereas to reproduce lumpy features with a single scale, where the lumps have a characteristic length scale, Gaussian process emulators are highly effective, but may require large training sets to resolve the peaks. The quality of an emulator can be assessed through the following criteria:

- $E(\vec{\theta}_t) = M(\vec{\theta}_t)$ at the training points, $\vec{\theta}_t$.
- The emulator should reasonably reproduce the model away from the training points. This should hold true for either interpolation or extrapolation.
- The emulator should reasonably represent its uncertainty
- A minimal number of training points should be needed
- The method should easily adjust to larger numbers of parameters, $\theta_i,\ i = 1 \cdots N$
- The emulator should not be affected by rotations or translations of the parameter space
- The emulator should be able to account for noisy models (at which point the prediction would not necessarily exactly reproduce the training data)
- Training and running the emulator should not be numerically intensive

Here the goal is to focus on a particular class of functions: functions that are *smooth*. Smoothness is a prior knowledge of the function. It is an expectation that the linear terms of the function are likely to account for more variance than the quadratic contributions, which are in turn likely to be more important than the cubic corrections, and so on.

3.2 Mathematical Form of *Smooth Emulator*

Before proceeding, it has been assumed that the model is a function of model parameters, $\vec{X}$, where each dimension has its own prior. For the purposes of the theory, and for the purposes of calculation, these parameters are scaled and translated so that each X_i is transformed into a scaled coordinate θ_i. After translating the coordinates, the priors for the scaled coordinates are all centered at $\theta_i = 0$. If the initial prior is flat, after scaling the prior for θ_i is $-1 < \theta_i < 1$. If the prior has a Gaussian form, then the prior for the scaled variable is proportional to $e^{-3\theta_i^2/2}$. With this choice the variance for Gaussian and uniform priors are both equal to $1/3$ in the $\vec{\theta}$ coordinates.

To satisfy the criteria above, the functional form of the emulator in $N_{\rm pars}$ dimensional parameter space, $\theta_1 \cdots \theta_N$, is

$$Y(\vec{\theta}) \;=\; e^{-|\vec{\theta}|^2/2\Lambda^2} \sum_{\vec{n}} \frac{A_{\vec{n}}}{\sqrt{n_1! \cdots n_N!}} \left(\frac{\theta_1}{\Lambda}\right)^{n_1} \cdots \left(\frac{\theta_1}{\Lambda}\right)^{n_N}, \tag{3.1}$$

where, in the absence of training, the assumed-to-be-independent coefficients $A_{\vec{n}}$ are assumed to have a Gaussian distributions,

$$P(A_n) \;=\; \frac{1}{(2\pi\sigma_A^2)^{1/2}} e^{-A_n^2/2\sigma_A^2}. \tag{3.2}$$

Here, Λ and σ_A are what is known as hyper-parameters, i.e. they are not parameters of the full-model, but are parameters that define the emulator. The hyper-parameter Λ will be referred to as the convergence parameter and defines the expected relative strengths of various orders in the Taylor expansion. The hyper-parameter σ_A describes the characteristic strength of the coefficients. The coefficients $\vec{A}_{\vec{n}}$ are not referred to as hyper-parameters, but in practice the hyper-parameters $\vec{A}_{\vec{n}}$, σ_A and Λ are all treated similarly, in that they are all required to define the training data, and they are all allowed to vary according to some prior. Whereas the prior for the coefficients $A_{\vec{n}}$ is defined in Eq. (3.2), the priors for σ_A and Λ are generally assumed to be flat, though that can be changed. For example, one might simply state that Λ is constrained to be within some window, or might even be fixed.

With these choices for the mathematical form, the correlation between two points is

$$\begin{aligned} \langle Y(\vec{\theta}_1) Y(\vec{\theta}_2) \langle \;&=\; \sigma_A^2 C_0(\vec{\theta}_1, \vec{\theta}_2), \\ C_0(\vec{\theta}_1, \vec{\theta}_2) \;&=\; e^{-|\vec{\theta}_1 - \vec{\theta}_2|^2/2\Lambda^2}, \end{aligned} \tag{3.3}$$

where the averaging refers to an average over the coefficients $A_{\vec{n}}$. Rather than averaging over the coefficients, one could maximize the likelihood, but the resulting expression is identical. One can find the most likely set of coefficients that reproduces the values of the full model runs for a given σ_A and Λ. This leads to an optimized prediction

$$Y_{\rm opt}(\vec{\theta}) \;=\; \sum_{ab} C_0(\vec{\theta}, \vec{\theta}_a) B_{ab}^{-1} y_b, \tag{3.4}$$

$$B_{ab} \;=\; C_0(\vec{\theta}_a, \vec{\theta}_b) + \alpha^2 \delta_{ab}. \tag{3.5}$$

Here, y_a are the values of the full model calculations measured at the training point $\vec{\theta}_a$, and α is a measure of the point-by-point noise. If one thinks that the calculated values vary randomly by one percent of the characteristic values, one would set $\alpha = 0.01$. Remarkably, this is exactly the same expression as for a Gaussian process (GP) emulator with a quadratic kernel. This derives from the fact that any two distributions with the same correlation result in the same emulator, a.k.a the 'kernel trick'. The emulator prediction in Eq. (3.4) is independent of σ_a, but depends on Λ.

The emulator's uncertainty (for fixed σ_A and Λ) is:

$$\langle \delta Y(\vec{\theta})^2 \rangle \;=\; \sigma_A^2 \left[1 - \sum_{ab} C_0(\vec{\theta}, \vec{\theta}_a) B_{ab}^{-1} C_1(\vec{\theta}_b, \vec{\theta}) \right]. \tag{3.6}$$

This expression assumes fixed σ_A and Λ. This expression ignores the contribution from the uncertainties in the hyper-parameters. Including those uncertainties is described below.

The likelihood distributions for σ_A and Λ are given by the function

$$\begin{aligned} Z(\sigma_A, \Lambda) &= \int \prod_{\vec{i}} P(\vec{A}, \sigma_A, \Lambda) \\ &= \sigma_A^{-N} (\det B)^{-1/2} \exp\left[-\frac{1}{2\sigma_A^2} y_a B^{-1}_{ab} y_b\right], \end{aligned} \tag{3.7}$$

where $P(\vec{A}, \sigma_A, \Lambda)$ is the prior probability constrained by the training points. To calculate the uncertainties, *Smooth Emulator* uses the maximum likelihood values of σ_A and Λ, but when including the additional emulator uncertainty due to the fact that σ_A and Λ vary, *Smooth Emulator* uses the curvature of $Z(\sigma_A, \Lambda)$. The uncertainty for the emulator is then

$$\begin{aligned} \langle \delta Y(\vec{\theta})^2 \rangle &= \sigma_A^2 \left[1 - \sum_{ab} C_0(\vec{\theta}, \vec{\theta}_a) B^{-1}_{ab} C_0(\vec{\theta}_b, \vec{\theta})\right] \\ &\quad + W_{\sigma_A \sigma_A} \left(\frac{\partial Y}{\partial \sigma_A}\right)^2 + 2 W_{\sigma_A \Lambda} \left(\frac{\partial Y}{\partial \sigma_A}\right)\left(\frac{\partial Y}{\partial \Lambda}\right) + W_{\Lambda\Lambda} \left(\frac{\partial Y}{\partial \Lambda}\right)^2. \end{aligned}$$

Here, $W_{\alpha\beta}$ are the derivatives of $\ln(Z)$. All derivatives are calculated at the point of maximum likelihood. One can find a detailed explanation of these expressions in `smoothdraft.pdf`, including analytic expressions for W and for $\partial Y / \partial \alpha$.

From here on, the function $Y(\vec{\theta})$ and the uncertainties will refer to expressions where the coefficients $A_{\vec{n}}$ are chosen to optimize the likelihood for a given σ_A and Λ, and where σ_A and Λ were chosen to optimize $Z(\sigma_A, \Lambda)$.

3.3 Training-Point Optimization

Training-point locations should be chosen to minimize the uncertainty defined in Eq. (3.8) integrated over the prior. If the hyper-parameters are labeled by some vector $\vec{h}$, one would like to minimize

$$\langle\langle \delta Y^2 \rangle\rangle = \int d\vec{h}\ P(\vec{h}) \langle \delta Y^2 \rangle(\vec{h}). \tag{3.8}$$

However, one must choose training points (at least the first set) before one has actually run the full model. The last several terms in Eq. (3.8) depend on the values of the full model at the training points, y_a, or more precisely on the matrix $y_a y_b$. This can be replaced by the prior expectation, $y_a y_b \to \sigma_A^2 C_0(\vec{\theta}_a, \vec{\theta}_b)$. The resulting estimate in Eq. (3.8) is the product of σ_A^2 multiplied by a complicated function of Λ and the locations of the training points. One does not know, a priori, either σ_A or Λ. Fortunately the value of σ_A^2 is irrelevant in determining the optimized positions. However, the optimized positions does depend on the value of Λ. Thus, in choosing the training points, one needs to first choose a value for Λ. Fortunately, from some studies, the choice of best positions is not strongly sensitive to Λ, i.e. if one chooses $\Lambda = 2.5$ vs. $\Lambda = 3$, the resulting positions vary rather little.

Once one has chosen a reasonable value for Λ one can fortunately find analytic expressions for $\langle\langle \delta Y^2 \rangle\rangle / \sigma_A^2$. The expressions, which are provided in smoothdraft.pdf, are rather messy, so finding the optimized points involves a search through all possible $N_{\rm pars}$ dimensional positions of $N_{\rm train}$ points. In *Smooth Emulator* this is accomplished by a simple steepest decent algorithm. The procedure takes little time when $N_{\rm pars} \lesssim$ a dozen, but can begin to require minutes or even a few hours for $N_{\rm pars} \gtrsim$ two dozen.

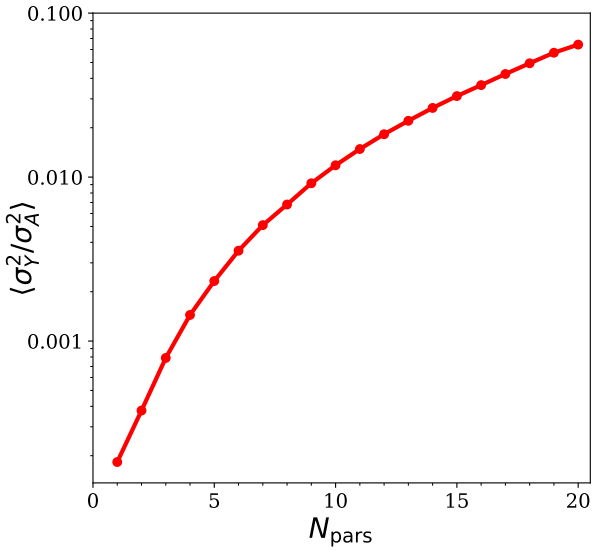


Figure 3.1: The accuracy metric as a function of the number of model parameters assuming Gaussian priors. The convergence parameter was set to $\Lambda = 3$, the point-by-point noise was set to $\alpha = 0.01$ and the number of training points was set to match the number of expansion coefficient of quadratic order or less. This illustrates the difficulty to accurately emulate higher dimensional functions even if the number of training points increases as $N_{\rm train} = (N_{\rm pars}+1)(N_{\rm pars}+2)/2$.

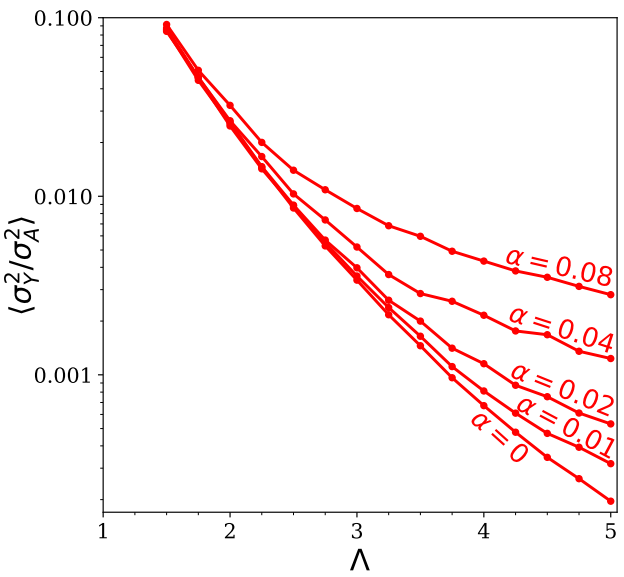


Figure 3.2: The expected accuracy of the emulator falls rapidly with the convergence parameter Λ. For this illustration $N_{\rm pars} = 6$ and the number of training points was set to 28, the number of expansion coefficients of order $n = 2$ or less. Calculations were repeated for different values of the point-by-point noise, α. As expected, the impact of the point-by-point noise sets in as the accuracy of the emulator improves.

Figure 3.1 shows the expected accuracy as a function of the number of model parameters. Even if the number of training points rises quadratically with the number of parameters (as shown in the figure), the expected accuracy rises substantially with $N_{\rm pars}$. A detailed discussion of the pernicious nature of higher dimensions can be found in smoothdraft.pdf. The accuracy strongly improves for higher convergence parameters, Λ, as shown in Fig. 3.2. More training points leads to better accuracy, as shown in Fig. 3.3. The expected uncertainty shown in Fig. 3.3 has noticeable kinks in the behavior. The number of parameters at which these kinks occur correspond to the number of parameters required for linear, quadratic, cubic fits $\cdots$. For linear fits, that number is $N_{\rm pars}+1$, for quadratic it is $(N_{\rm pars}+1)(N_{\rm pars}+2)/2$ and for cubic fits, $(N_{\rm pars}+1)(N_{\rm pars}+2)(N_{\rm pars}+3)/6$. Given the kinks, it might behoove the User to choose a number of training points corresponding to the locations of these kinks. Finally, Fig. 3.4 shows the expected accuracy as a function of the number of steps in the steepest descent algorithm. Because the algorithms can start with either a random placement of training points, or with a selection using a Latin hyper-cube algorithm, one can see the expected benefit of this optimization procedure relative to simply choosing the points randomly

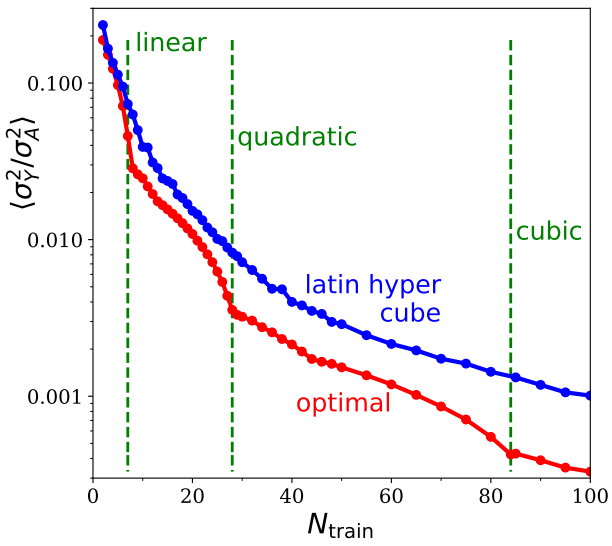


Figure 3.3: For all Gaussian priors, with $N_{\text{pars}} = 6$, $\Lambda = 3$, and $\alpha = 0.01$, the expected mean accuracy falls with increasing the number of training points. The minimum number of training points required for linear, quadratic and cubic fits are 7, 28 and 82. After passing one of these threhsolds the marginal improvement for additional training points decreases. For $N = 8$, one can place the points in a simplex with an additional point at the center, and the symmetry of this configuration provides an especially large improvement. This behavior might inspire the modeler to choose a number of training points equal to one of the thresholds. The optimized placement significantly improves compared to Latin hyper-cube sampling.

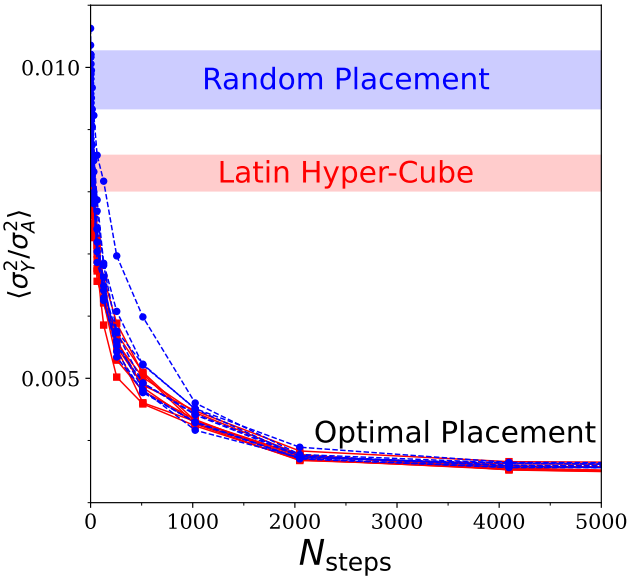


Figure 3.4: The search procedure provides most of the improvement for the first few thousand steps. For this example, $N_{\text{pars}} = 6$, $\Lambda = 3$ and $\alpha = 0.01$ the number of training points was set to 28. Several initial configurations were considered. Those where the points were placed randomly (blue circles) tended to start at roughly three times the optimized value, whereas those that were initialized with Latin hyper-cube sampling tended to start at a bit less than double the optimized uncertainty. The bands roughly represent the spread of initial accuracies, and the fall from the bands to the optimal values represents the improvement one expects to gain through the optimization procedure.

or using Latin hyper-cube sampling. Typically the improvement is on the order of a factor of two, with larger relative improvements for more parameters or more training points. This improvement can also be expressed by stating how many fewer training points one would need to consider to achieve the same accuracy. Roughly, this might be approximately a third fewer points for most cases.

In *Smooth Emulator* the optimization procedure first reads the file describing the prior model-parameter distribution. The prior distribution is assumed to an independent product of priors for each model parameter. Each distribution is assumed to be either a Gaussian, defined by a location and width, or a uniform distribution within some minimum and maximum values. *Smooth Emulator* scales all the model parameters to variables centered at the origin and with variances of 1/3. I.e., the scaled model-parameters have prior distributions that are either proportional to $e^{-3\theta_i^2/2}$ or uniform within the range $-1 < \theta_i < 1$. If one expected variation of the observable to be similar with respect to changing any of these variables, it would make sense to apply a single value of Λ for all N_{pars} model parameters.

Given the strong dependence on the number of parameters and on Λ, the User might wish to consider higher values of Λ for those model parameters expected to result in less variation of the observables within the prior. In *Smooth Emulator* this is accomplished by changing the widths of the priors, rather than by assigning different values of Λ. One chooses a single value of Λ to represent the convergence of the most sensitive model parameters. One then reduces the widths of the priors for the less important model parameters. Each model parameter is assigned a sensitivity, s_i. The priors for Gaussian and uniform distributions are then set to $e^{-3\theta_i^2/2s_i^2}$ or to $-s_i < \theta_i < s_i$. In practice, this is identical to assigning different values of Λ to each parameter scaled as $1/s_i$. This can be especially if there are large numbers of weakly sensitive parameters. For example, if one had 20 model parameters where the appropriate value of Λ was 2.5 for 5 parameters, but the appropriate value for the majority was closer to 4, one would set $\Lambda = 2.5$, then set $s_i = 1$ for the handful of more important parameters, and set $s_i = 5/8$ for the others. The optimization procedure selects training points to best capture the variation due to the first five parameters and worries less about positioning the points to capture higher-order polynomial behavior in the other parameters. By choosing $s_i < 1$ for several of the parameters the final estimate of the emulator's uncertainty should be significantly reduced.

4 Generating Training Points

4.1 Overview

The executable `trainingpoint_optiizer`, found in `${SMOOTH_HOME}/bin/` produces a list of points in the $n-$dimensional model-parameter space to be used for training an emulator. The points are chosen to minimize the estimate of the expected emulator uncertainty integrated over the model-parameter prior. Because the full model values are not determined at the time the training points are chosen, the estimate is based on the User's best guess for the convergence hyper-parameter Λ.

The program first reads a simple text file that provides the names of model parameters, the range of their prior distribution and their expected relative sensitivities, `smooth_data/Info/prior_info.txt`. Options for `trainingpoint_optimizer` are taken from a separate text file, `smooth_data/Options/tpo_options.txt`. This enables the User to make choices, such as which algorithm to apply when generating the training points. For some algorithms, `trainingpoint_optimizer` chooses the number of training points, whereas for other options the User chooses the number. Before the emulator is built and tuned, the User is free to run the full model at additional points, or to use their own method to generate training points and forego `trainingpoint_optimizer` entirely.

4.2 Specifying Model Parameters and Priors

Before proceeding, the executable `trainingpoint_optimizer` requires information about the parameters, specifically, their ranges. The User enters this information into the file `smooth_data/Info/prior_info.txt`, as described in Chapter 2. An example of such a file might be

```
# par_name dist_type  xmin/centroid  xmax/Rgauss  SensitivityScale
   parameter_name_0 prior_type_0 xmin_0/centroid_0 xmax_0/width_0 s_0
   parameter_name_1 prior_type_0 xmin_1/centroid_1 xmax_1/width_1 s_1

   parameter_name_n prior_type_n xmin_n/centroid_n xmax_n/width_n s_n
```

The first column is the model-parameter name, and the last three parameters describe the range of the parameters in the prior, assuming the prior is uniform or Gaussian. The second entry for each parameter defines whether the prior range is `uniform` or `gaussian`. If the prior is `uniform`, the next two numbers specify the lower and upper ranges of the parameter. If the prior range is `gaussian`, the third entry describes the center of the Gaussian, x_0, and the fourth entry describes the Gaussian width, σ_0, where the prior distribution is $\propto \exp\{-(x-x_0)^2/2\sigma_0^2\}$. `trainingpoint_optimizer` will read the information to determine the number of parameters. It will then translate and scale the model parameters from x_i to θ_i so that the n values of θ, $\theta_{1\cdots n}$, have their priors centered at the origin with variances of $s_i^2/3$.

`training_point_optimizer` takes all this information into account when optimizing the selection of points. For example if the sensitivity scale for a given model parameter is small, points will be chose so better capture behavior along the other parameters. Chapter 3 provides a brief description

of the theoretical basis of the training-point selection and for a more detailed description one can read smoothdraft.pdf.

4.2.1 Format for Storing the Coordinates of the Training Points

The information for training is all stored in a directory. The default directory is `smooth_data/FullModelRuns/`, but that directory can be changed by setting options (see below). With that directory there are subdirectories `run0/,run1,`$\cdots$. Each subdirectory contains the information relative to the full-model run at a specific point in model-parameter space. Within each run directory, there are two files, one describing the model-parameters, `smooth_data/FullModelRuns/runI/model_parameters.txt`, and a second text file describing the values of the observables, `smooth_data/runI/obs.txt`, at that point in parameter space. The format for the `model_parameters.txt` files is:

```
parameter_name_1 x1
parameter_name_2 x2
.
```

The model-parameter names must be identical to those listed in `smooth_data/Info/prior_info.txt`, and the values of those parameters are `x1`$\cdots$. The training-point optimization software does not require information about the observables.

4.3 Training Point Optimizer (TPO) Options

These options are all set in the ascii file `smooth_data/Options/tpo_options.txt`, where the path is relative to the analysis directory. An example of the file is:

```
#LogFileName                tpolog.txt  # comment out to direct output to screen
TPO_Method  MCQuadratic  # can be "MC", "MCSphere", "MCSimplex",
↪  "MCSimplexPlus1", "MCQuadratic", "MCSphereQuadratic"
TPO_NTrainingPts    0   # only relevant for OptimizeMethod="MCSphere" or "MC"
TPO_NMC  10000
TPO_ALPHA 0.01
TPO_LAMBDA 2.5
TPO_Include_LAMBDA_Uncertainty true
TPO_FullModelRunsDirName  smooth_data/FullModelRuns
TPO_RANSEED 12345
#TPO_ReadPoints 0,1,4-7,10,11-13,21
#TPO_FreezePoints 0,1,4-7,10,11-13,21
```

For each line the first string is the option name and is followed by the value. Both are single strings (without spaces). The # symbol is used for comments. Each parameter has a default value, which will be used if the parameter is not mentioned in the parameter file. The paths are provided relative to the analysis directory, i.e. the directory from which you run the `trainingpoint_optimizer` command. The various options control the behavior as described below.

4.3.1 `LogFileName`

If this is left blank, *Training Point Optimizer* will write output to the screen. Otherwise it will write output to the named file. Given that *Training Point Optimizer* runs in a few seconds, the program is usually run interactively and output is sent to the screen.

4.3.2 `TPO_LAMBDA` (default is `2.5`)

The locations of the training point depends on a guess for the convergence hyper-parameter Λ. Because emulator optimization cannot be performed before the model is run, the User must provide a guess at this point. The dependence of the locations on Λ is somewhat weak, so there is little need to worry too much about this. However, the actual accuracy of the emulator depends strongly on Λ, so once the emulator is built the actual uncertainty of the emulator can significantly differ from the predictions provided by the training-point-optimization program

4.4 `TPO_Include_LAMBDA_Uncertainty` (default is `true`)

In minimizing the predicting the uncertainty the optimization considers the uncertainty due to variation of the choice of `LAMBDA`. Leaving this out can lead to rather extreme choices for the positions, so `true` is highly recommended.

4.4.1 `TPO_Method` (default `MC`)

The option `TPO_Method` sets the training point method and can be set to the following values:

- `MC`: This is the default. It uses a Monte Carlo procedure to adjust all the training-point locations to optimize the metric defining the best placements. This should be the choice for
- `MCSphere`: This constrains all the points, except for one, to be located on the surface of a sphere. The last point is placed in the center. The radius of the sphere is adjusted to optimize the metric.
- `MCSimplex`: This places all the points at the same radius and equally spaced from each other. The number of training points for this option is automatically set to $N_{\rm train} = N_{\rm par} + 1$. Examples of simplexes are: an equilateral triangle for $N_{\rm par} = 2$ and a tetrahedron for $N_{\rm par} = 3$. If one is choosing the number of training points to correspond to the minimum number required to fix a linear fit, and if the prior is spherically symmetric (e.g. gaussians with the same sensitivity scales), then this option is precisely the optimum one.
- `MCSimplexPlus1`: The same as above, but adding one point in the center. The number of training points is then set to $N_{\rm train} = N_{\rm par} + 2$.
- `MCQuadratic`: This is the same as the `MC` option except that the number of training points is fixed at $N_{\rm train} = (N_{\rm par} + 1)(N_{\rm par} + 2)/2$, which is sufficient to fix all the coefficients in a quadratic expansion.
- `MCSphereQuadratic`: This is the same as `MCSphere`, except that the number of training points is set to $N_{\rm train} = (N_{\rm par} + 1)(N_{\rm par} + 2)/2$.

4.4.2 TPO_NTraining_Pts (default 0)

For the MC and MCSphere methods the program chooses $N_{\rm train}$, the number of training points. Otherwise $N_{\rm train}$ is set to correspond to the method as described above.

4.4.3 TPO_ALPHA (default 0.01)

The value ALPHA represents the point-to-point noise, which might vary from one observable to another. But the same training points are used for all observables. Thus, choosing ALPHA can be not wholly consistent with those values defined for the observables. Fortunately, because the training points will be chosen far apart, this option has little impact. But, in choosing the points making this non-zero helps protect against extreme configurations with points close together. The value of 0.01 works well. This value does not need to equal the values used to train the emulator (which can vary from one observable to another).

4.4.4 TPO_FullModelRunsDirName (default smooth_data/FullModelRuns)

This sets the directory into which the full model output for the training points will be stored. A similar parameter is used by the emulator and is set in smooth_data/Options/emulator_options.txt, so the User should be careful that if the output from the optimizer is changed in a different location that the emulator also knows where to find it.

4.4.5 TPO_RANSEED (default 12345)

Sets seed for random number generator.

4.4.6 TPO_ReadPoints (default NONE)

If set to NONE, the MC procedure will be initialized according to latin hyper-cube sampling. If set to ALL, the procedure initializes the search using the training point locations described in the smooth_data/FullModelRuns/runI/model_parameters.txt files. This is useful if one has a better idea of how to initialize the points, or if the point is to be frozen at a specific location as defined by the next option below. If the User wishes to read in a subset of points, the string can list those points which will be read in, while the others will be initialized randomly. An example of the string is 0-2,5,21-23.

4.4.7 TPO_FreezePoint (default NONE)

If set to NONE all the training points are allowed to vary in the optimization procedure. If set to a string, e.g 0-2,5,21-23, the procedure will freeze those points denoted in the string. These points should be a subset of the those points that are read in as described above.

4.5 Running `trainingpoint_optimizer` to Generate Training Points

To run *Training Point Optimizer*, first make sure the program is compiled. To compile the programs, change into the `${SMOOTH_HOME}/software/MainPrograms/` directory and enter the following command,

```
${SMOOTH_HOME}/software% cmake .
${SMOOTH_HOME}/software% make trainingpoint_optimizer
```

Next, change into your analysis directory and run the program.

```
${MY_ANALYIS}% ${SMOOTH_HOME}/bin/trainingpoint_optimizer
```

Here `${SMOOTH_HOME}/bin` is the path to where the User compiles the main programs into executables.

`trainingpoint_optimizer` will read parameters from the `smooth_data/Options/tpo_options.txt` file and from the `smooth_data/Info/prior_info.txt` files. It will then write the information about the training points in the directory `smooth_data/FullModelRuns/`. Within the directory, a sub-directory will be created for each training point, named `run0/, run1/, run2/`$\cdots$. Within each subdirectory, `trainingpoint_optimizer` creates a file `runI/model_parameters.txt` for the I^{th} training point. For example, the `run0/model_parameters.txt` file might be

```
NuclearCompressibility     229.08
ScreeningMass              0.453
Viscosity                  0.192
```

At this point, it is up to the User to run their full model at each training point and create a file `runI/obs.txt`, which stores values of the observables at those training points as calculated by the full model.

4.6 Using Other Choices (other than from *Training Point Optimizer*) for Training Points

If the User desires to use their own procedure for training points, or if the User wishes to augment the *Training Point Optimizer* points with additional training points, the User can write their own files `runI/model_parameters.txt`. The emulator should work fine, though the User should remember that when training the emulator (see Chapter 6) the emulator parameter describing which `runI` directories to apply should be modified.

There is one warning to be issued when choosing training points for tuning *Smooth Emulator*. You can get an error if some training point does not provide independent information from the other points, which then leads to singularities in the matrix algebra. For example, if two of the training points were exactly the same, it would fail. In a sense, the `trainingpoint_optimizer` program chooses training points to stay as far away from this possibility as possible.

4.7 Writing Programs Accessing the Training Point Optimization Software

Smooth Emulator Software is organized to accommodate the User building their own main programs that incorporate the functionality. For the training-point optimization software, the software's functionality is rather simple and the User can adjust the functionality through setting options as described above. The main programs and the executables are stored in the `${SMOOTH_HOME}/software/MainPrograms/` and `${SMOOTH_HOME}/bin/` directories respectively. The provided main program's source code, `trainingpoint_optimizer_main.cc`, aside from the headers, is:

```
int main(){
   NBandSmooth::CTPO *tpo=new NBandSmooth::CTPO();
   tpo->Optimize();
   tpo->WriteModelPars();
   return 0;
}
```

Several additional methods can be seen by viewing the header file, `${SMOOTH_HOME}/software/include/msu_smooth/trainingpoint_optimizer.h`. However, the methods typically called are `Optimize()` and `WriteModelPars` with the needed flexibility provided by choosing the appropriate options.

4.7.1 The `parameterMap` utility class

If the User wishes to write programs that change options during the running, they can use the parameterMap class, which has an object within the `CTPO` class, `tpo->parmap`. For example, the User could set `LAMBDA=3.0` by adding the following line to the main program:

```
   .
tpo->parmap.set("TPO_LAMBDA",3.0);
   .
```

If the User wanted to have the program find the value, they could add a line, for example,

```
   .
int NMC=tpo->parmap.getI("TPO_NMC",0);
   .
```

Further description of the `parameterMap` class is provided in Sec. 9.1.

5 Performing Full Model Runs

5.1 Overview

After setting the locations of the training points in model-parameter space, as described in Chapter 4, it is the responsibility of the User to run the full model at those points and record the values of all the observables for each point. Only then can the *Smooth Emulator* software proceed to the next step of tuning the emulator. Training data for a specific model run generated from a specific point in model-parameter space is stored in a unique directory.

5.2 Format for Training Data

Once the training points are generated, the User must run the full model for each of the training points. Before running the full model, there is a directory, `${MY_ANALYSIS}/smooth_data/FullModelRuns/`, in which there are sub directories, `run0/, run1/, run2/....` Within each sub-directory, `runI`, there should exist a text file `smooth_data/FullModelRuns/runI/model_parameters.txt`, where $I = 0, 1, 2, \cdots$. These files could have been generated by `trainingpoint_optimizer`, but could have been generated by any other means, including by hand. The files should be of the form,

```
parameter_name_1  x1
parameter_name_2  x2
parameter_name_3  x3
.
```

The model-parameter names must match those defined in `${MY_ANALYSIS}/smooth_data/Info/prior_info.txt`, the format of which is described in Chapter 4.

The User must then perform full model runs using the model-parameter values as defined in each sub-directory. The full model runs should write a list of observable values in each run directory in files named `smooth_data/FullModelRuns/runI/obs.txt`, where $I = 0, 1, 2, \cdots$. The format of those text files should be

```
observable_name_1  Y1
observable_name_2  Y2
observable_name_3  Y3
.
```

The names must match those listed in `smooth_data/Info/observable_info.txt`, as described in Chapter 6. The observable values are calculated by the full model for the model-parameter values listed in the corresponding `model_parameters.txt` file in the same directory.

Once the observable files are produced for each of the full model runs, the User can then proceed to build and tune an emulator.

6 Tuning the Emulator

6.1 Overview

Smooth Emulator effectively finds an optimum set of polynomial (or Taylor) expansion coefficients, that reproduce a set of observables at a set of training points. The process of finding those coefficients, given the training data, is referred to as "tuning". For a given observable, a particular sample set of coefficients assumes the following functional form for the emulator:

$$Y(\vec{\theta}) = \sum_{\vec{n}} f(|\vec{\theta}|) d(\vec{n}) A_{\vec{n}} \left(\frac{\theta_1}{\Lambda}\right)^{n_1} \left(\frac{\theta_2}{\Lambda}\right)^{n_2} \cdots . \tag{6.1}$$

Here, $\theta_1\theta_2\cdots$ represent the original model parameters, $\vec{X}$, but are scaled. If their initial prior is uniform, they are scaled so that their priors range from -1 to +1, and if they have Gaussian priors, they are scaled so that their variance is one third to match that of the flat prior. The coefficients are assumed to all follow a Gaussian distribution,

$$P(\vec{A}) = \prod_n \frac{1}{\sqrt{2\pi\sigma_A^2}} e^{-A_n^2/2\sigma_A^2}. \tag{6.2}$$

The hyper-parameter Λ is the "convergence" parameter because it sets the relative strength of terms of different order. Higher values of Λ lead to expansions that converge more quickly. The second hyper-parameter, σ_A, simply sets the overall strength, and variance, of the observable throughout the prior.

Smooth Emulator chooses the envelope function, $f(|\vec{\theta}|)$, and the degeneracy factor, $d(\vec{n})$, so that the variance of the non-trained emulator is independent of $\vec{\theta}$, and so that correlations depend only on relative $\vec{\theta}$. With these choices

$$\begin{aligned} d(\vec{n}) &= \sqrt{\frac{(n_1+n_2+\cdots)!}{n_1!n_2!\cdots}}, \\ f(|\vec{\theta}|) &= e^{-|\vec{\theta}|^2/2\Lambda^2}. \end{aligned} \tag{6.3}$$

For more details the User can look at Chapter 3, or at the included draft, `smoothdraft.pdf`.

Due to the kernel trick *Smooth Emulator* needn't calculate all the coefficients, even though there are expressions for them. Instead the resulting form for the optimum emulation is

$$\begin{aligned} Y_{\rm opt}(\vec{\theta}) &= \sum_{ab} C_0(\vec{\theta},\vec{\theta}_a) B^{-1}_{ab} y_b, \\ B_{ab} &= C_0(\vec{\theta}_a,\vec{\theta}_b) + \alpha^2\delta_{ab}, \\ C_0(\vec{\theta}_1,\vec{\theta}_2) &= e^{-|\vec{\theta}_1-\vec{\theta}_2|^2/2\Lambda^2}. \end{aligned} \tag{6.4}$$

Closed form for the uncertainties, as provided in `smoothdraft.pdf`, are also calculated by *Smooth Emulator* software. Here α represents an estimate of the point-by-point uncertainty as a fraction of α. If one thinks the model calculations have an uncertainty that due to sampling or noise varies by an amount $\sigma_{\rm noise}$, and if one believes the characteristic strength of Y is σ_A, one chooses $\alpha =$

$\sigma_{\rm noise}/\sigma_A$. For example, if the full-model uses some sampling algorithm that has a 1% uncertainty, one would choose $\alpha = 0.01$. More discussion is provided in Chapter 3. Remarkably, this form exactly reproduces that of a GP emulator, if the GP emulator chooses a kernel with a Gaussian form. This equivalence follows from the fact that, due to the kernel trick, the emulator's predictions depend only on $C_0(\vec{\theta}_1, \vec{\theta}_2)$ and α.

Given the training data, *Smooth Emulator* finds a best estimate of the two hyper-parameters. From `smoothdraft.pdf` one can see that the choice of the two hyper-parameters for very similar functions can vary substantially, because there is weak sensitivity to the ratio Λ/σ_A, and strong sensitivity to the product $\sigma_A\Lambda$. Nonetheless, the emulator does a good job in predicting its uncertainty.

The executables based on *Smooth Emulator* are located in the User's `${SMOOTH_HOME}/bin` directory. Examples of such executables are `smoothy_tune` or `smoothy_calcobs`. These functions must be executed from within the User's analysis directory. In the following subsections, we first review the format for each of the required input files, then describe how to run *Smooth Emulator*, how its output is stored.

6.2 Preparing Files for *Smooth Emulator*

Before training the emulator, one must first run the full model at a given set of training points. In addition to an options file (described in the next sub-section), the User must provide the following:

I. A file listing the names of observables. This file is named `smooth_data/Info/observable_info.txt`, where the path is relative to the analysis directory. The file format is:

```
observable_name_1   ALPHA_1
observable_name_2   ALPHA_1
.
observable_name_n   ALPHA_N
```

The observable names should be simple strings, without spaces. Here, the values of `ALPHA_i` represent the point-by-point uncertainties inherent to the full-model calculations. If there is no such uncertainty, i.e. if you reran the full model with the same model parameters, you would always get the exact same value, then the values for `ALPHA_i` should be zero. If you think the point-by-point uncertainty is σ_{pp}, and given that the typical strength of the function is σ_A, one would set `ALPHA_i` to σ_{pp}/σ_A.

II. A file listing the names of the model parameters and their priors. This file is `smooth_data/Info/prior_info.txt`. The file format is:

```
parameter_name_1  prior_type_1 xmin_1/centroid_1  xmax_1/Rgauss_1  ss_1
parameter_name_2  prior_type_2 xmin_2/centroid_2  xmin_2/Rgauss_2  ss_2
.
parameter_name_N  prior_type_N xmin_N/centroid_N  xmin_N/Rgauss_N  ss_N
```

Again, the parameter names should be simple strings without spaces. The `prior_type_I` entries are character strings, either `uniform` or `gaussian`. If the prior type is `uniform`, `xmin_I` and `xmax_I` represent the endpoints of the uniform prior. If the prior type is `gaussian`, the

prior is assumed to have a Gaussian form, the two values set the centroid and widths of the Gaussian priors. The last column is the sensitivity scale. If all model parameters are expected to contribute equally to the overall variance throughout the prior, all the `ss_I` variables should be set to unity. If some model parameters are expected to cause less variance, then they should be set to some fraction of unity.

III. At each training point the User must provide a list of the model-parameter values, $\vec{x}$, and the values of each observable as calculated by the full model at $\vec{x}$. The values of $\vec{x}$ at each point can be generated by `trainingpoint_optimizer`, as described in Chapter 4, or they can be entered by hand. It is up to the User to run the full model at each training point to generate the observables.

The default location for files containing this information is the directory `smooth_data/FullModelRuns`, though that location can be changed in the options file described below. For each training point, the training-point and observable information is stored in a separate directory. If there are 100 training points, the directories would be `smooth_data/FullModelRuns/run0/`, `smooth_data/FullModelRuns/run0/`, $\cdots$, `smooth_data/FullModelRuns/run99/`.

Each directory, `runI`, must contain two files, `runI/model_parameters.txt` and `runI/obs.txt`. The file `model_parameters.txt` has the format:

```
parameter_name_1 x1
parameter_name_2 x2
.
```

The model-parameter names must be identical to those listed in `smooth_data/Info/prior_info.txt`, and the values of those parameters are `x1`$\cdots$.

After running the full model, the User creates the files `runI/obs.txt` in the following format.

```
observable_name_1 y1
observable_name_1 y2
.
```

6.3 *Smooth Emulator* Options

Smooth Emulator requires a file describing the options, `${MY_ANALYSIS}/smooth_data/Options/emulator_options.txt`. The ascii file is simply a list of option names followed by values. Here is an example of such a file.

```
# Location of output, comment out for interactive running
  # LogFileName smoothlog.txt
  SmoothEmulator_FixLambda false // If false will adjust optimize LAMBDA (false
  ↪  is default)
  # Smoothness parameter to start maximizing procedure
  SmoothEmulator_LAMBDA 2.5    // If SmoothEmulator_FixLambda is true, this fixes
  ↪  LAMBDA
  #
```

```
# Location of training data, default is smooth_data/FullModelRuns
SmoothEmulator_FullModelRunsDirName smooth_data/FullModelRuns
#
# Choose which runs to use for training. Can be set to "all" or as list, e.g.
 ↪  1-5,8-12,15,18
SmoothEmulator_TrainingPts all
#
# Below only used for testing agains full model runs not used for training.
# Default location of testing data is smooth_data/FullModelTestingRuns/
SmoothEmulator_FullModelTestingRunsDirName smooth_data/FullModelTestingRuns
#
# List of points to be used for testing, all is default, can list, e.g.
 ↪  1-5,8-12,15,18
SmoothEmulator_TestingPts all
#
# When calculating uncertainty, include(or not) the contribution from Lambda
 ↪  varying
SmoothEmulator_INCLUDE_LAMBDA_UNCERTAINTY true
#
```

Any line beginning with `#` is ignored. If any of these lines are missing from the options file, *Smooth Emulator* will assign the default value. Here is a synopsis of the various options one can set.

6.3.1 `LogFileName`

If this is commented out, as it is in the example above, *Smooth Emulator*'s main output will be directed to the screen and the program will run interactively. Otherwise, the output will be recorded in the specified file. Most often, one will wish the program to run interactively.

6.3.2 `SmoothEmulator_FixLambda (default false)`

Smooth Emulator will choose optimum values for the hyper-parameters Λ and σ_A based on the training data. But, if the User wishes to choose their own value of Λ, they can set `SmoothEmulator_FixLambda` to `false`.

6.3.3 `SmoothEmulator_LAMBDA (default 3.0)`

If `SmoothEmulator_FixLambda` is set to `true`, this sets the convergence parameter Λ.

6.3.4 `SmoothEmulator_FullModelRunsDirName (default smooth_data/FullModelRuns)`

This is only relevant if `SmoothEmulator_TrainingFormat` is set to `SMOOTH`. This identifies the location of the directory where the training data is stored. The various run directories, `run0`, `run1`, $\cdots$, are subdirectories of this directory, and each sub-directory stores the information defining the full-model run for a specific point in model-parameter space.

6.3.5 SmoothEmulator_TrainingPts (default all)

This is also only relevant if SmoothEmulator_TrainingFormat is set to SMOOTH. If set to the default value, all, all the full-model runs in the directory identified by SmoothEmulator_FullModelRunsDirName will be used to train the emulator. If one wishes to use some subset of the run directories, this should be a string featuring numbers, commas and dashes. For example, 0-3,9,12-14 uses the runs numbered 0,1,2,3,9,12,13,24.

6.3.6 SmoothEmulator_FullModelTestingRunsDirName (default smooth_data/FullModelRuns)

This is only relevant if SmoothEmulator_TrainingFormat is set to SMOOTH. This identifies the directory where the testing data is stored. The various run directories, run0, run1, $\cdots$, are subdirectories of this directory, and each sub-directory stores the information defining the full-model run for a specific point in model-parameter space.

6.3.7 SmoothEmulator_TestingPts (default all)

This is also only relevant if SmoothEmulator_TrainingFormat is set to SMOOTH. If set to the default value, all, all the full-model runs in the directory identified by SmoothEmulator_FullModelRunsDirName will be used to train the emulator. If one wishes to use some subset of the run directories, this should be a string featuring numbers, commas and dashes. For example, 0-3,9,12-14 uses the runs numbered 0,1,2,3,9,12,13,24.

6.4 Running *Smooth Emulator* Programs

The source code for several *Smooth Emulator* main programs can be found in the ${SMOOTH_HOME}/software/MainPrograms/ directory. They are separated from the bulk of the software, which is in the ${bandframework}/SmoothEmulator/software/ directory. The main programs are designed so that the User can easily copy and edit them to create versions that might be more appropriate to the User's specific needs. When compiled, from the ${SMOOTH_HOME}/software/ directory, the executables appear in the ${SMOOTH_HOME}/bin/ directory. If the User wishes to make new executables, it is perhaps easiest to edit and rename the existing programs in the ${SMOOTH_HOME}/software/MainPrograms/ directory and then add the appropriate line to the ${SMOOTH_HOME}/software/CMakeLists.txt file. After briefly discussing how to run the codes, a review of the four codes that use the emulation functionality is provided. The source code for these four codes is provided here with the purpose of showing the User how they might use these codes as bases for designing and coding their own main programs.

From within the ${SMOOTH_HOME}/software/ directory, one can compile the programs with the command:

```
${SMOOTH_HOME}/software% cmake .
${SMOOTH_HOME}/software% make
```

The executables should now appear in the ${SMOOTH_HOME}/bin/ directory. If one enters `make` *program name* only that program will be compiled. One can peruse the source codes in ${SMOOTH_HOME}/software/MainPrograms. Assuming the directory ${SMOOTH_HOME}/bin/ has been added to the User's path, the User may switch to the User's analysis directory, and enter the program name, e.g.

```
 ${MY_ANALYSIS}% smoothy_testattrainingpts
```

There are four main programs provided in the distribution which read in the training data, train an emulator and perform an operation. The four executables are:

6.4.1 smoothy_testattrainingpts

This program builds and tunes the emulator, then compares the emulator predictions to the training values. If the point-by-point noise parameter, α, is set to zero the emulator should precisely match the training values. The output of the program lists the predictions for each observables. For example, the output might look something like this:

```
% smoothy_testattrainingpts
 ------ first_observable_name -------------
 ---- Lambda=3.070954, SigmaA=93.242332
 - itrain --- Y_full     Y_emulator    Sigma_emulator
  0 Y[0]= 4.385e+01 =?  4.385e+01  +/-  2.41322e-05
  1 Y[0]= 9.320e+01 =?  9.320e+01  +/-  1.53834e-05
  2 Y[0]= 4.381e+01 =?  4.381e+01  +/-  2.57861e-05
  .
```

The first column denotes the specific training point, the second and third columns represents the observable as calculated by the full model and emulator, with the emulator's uncertainty provided in the last column. After printing the information for the first observable, the program then provides the comparison for the subsequent observables.

The main program, ${SMOOTH_HOME}/software/MainPrograms/smoothy_testattrainingpts_main.cc, for this executable is rather short and simple:

```
int main(){
   NBandSmooth::CSmoothMaster master;
   master.TuneAllY();
   master.TestAtTrainingPts();
   return 0;
}
```

The functionality is mainly accessed through the `CSmoothMaster` object. The constructor reads in the information and options files, plus the training data (and testing data if it exists). The tuning command, `master.TuneAllY()`, finds optimum values for the two hyper-parameters, plus stores a few arrays for quicker calculation.

6.4.2 smoothy_testvsfullmodel

As the name suggests, this program compares the emulated values to full-model calculations, but instead of comparing to values calculated at training points, it compares to full-model calculations at training points not used to train the data. For each observable, the output describes the accuracy averaged over the testing points. An example output is:

```
% smoothy_testvsfullmodel
 iY=0: <(Y-Yreal)^2>/SigmaY^2_emulator=1.313787
 percent < 1 sigma = 75.000000
 iY=1: <(Y-Yreal)^2>/SigmaY^2_emulator=1.304912
 percent < 1 sigma = 69.642857
 .
```

For the first observable, this shows that the average (over the testing points) difference squared between the emulator and full model for the first observable was 1.314 times the stated uncertainty and that 75% of the emulator predictions were within the stated emulator uncertainty. The User can find a training-point-by-training-point comparison in the file `smooth_data/output_stuff/fullmodel_testdata/YvsY_`*observable_name*`.txt`.

The source code is also brief.

```
int main(){
   NBandSmooth::CSmoothMaster master;
   master.TuneAllY();
   master.TestVsFullModel();
   return 0;
}
```

6.4.3 smoothy_calcobs

This program prompts the User for the model parameters, then prints out the predictions for all the observables. The source code, `${SMOOTH_HOME}/software/MainPrograms/smoothy_calcobs_main.cc`, provides an example how the User might write a program to access more specific output:

```
int main(){
   NBandSmooth::CSmoothMaster master;
   //master.ReadCoefficients();
   master.TuneAllY();

   //Create model parameter object to store information about a single set of
    ↪ model parameters
   NBandSmooth::CModelParameters *modpars=new NBandSmooth::CModelParameters(); //
    ↪ object describes single point
   modpars->priorinfo=master.priorinfo; // priorinfo stores the information about
    ↪ the priors
   // Print out the prior information
```

```
   master.priorinfo->PrintInfo();

   // Prompt user for model parameter values and enter them into the modpars
    ↪ object
   vector<double> X(modpars->NModelPars);
   for(unsigned int ipar=0;ipar<modpars->NModelPars;ipar++)
       cout << "Enter value for " << master.priorinfo->GetName(ipar) << ":
       n";
       cin >> X[ipar];

   modpars->SetX(X);

   //  Calc Observables
   NBandSmooth::CObservableInfo *obsinfo=master.observableinfo;
   vector<double> Y(obsinfo->NObservables); // observable values
   vector<double> SigmaY(obsinfo->NObservables); // uncertainties
   master.GetAllY(modpars,Y,SigmaY); // Calculates Y and SigmaY in terms of the
    ↪ model parameters
   cout << "---- EMULATED OBSERVABLES ------
n";
   for(unsigned int iY=0;iY<obsinfo->NObservables;iY++)
       cout << obsinfo->GetName(iY) << " = " << Y[iY] << " +/- " << SigmaY[iY] <<
       endl;


   return 0;
}
```

Again, this source code illustrates how the User can access the functionality of the emulator through member functions of the `CSmoothMaster` class.

6.4.4 `smoothy_mcmc`

As the title suggests, this program runs the MCMC exploration of model-parameter space using the emulator. The source code for the included main program, `${SMOOTH_HOME}/software/MainPrograms/smoothy_mcmc_main.cc`, uses the functionality of both the `CSmoothMaster` class and the `CMCMC` class. The MCMC codes are discussed in detail in Chapter 7.

6.5 Functions and Methods of the `CSmoothMaster` and Related Classes

Smooth Emulator was designed so that the User can write their own main programs and access the functionality mainly through references to the `CSmoothMaster` object. Additionally, the User might find it useful to access the `CModelParameters`, `CparameterMap`, `CPriorInfo`, and `CObservableInfo` objects. Here is a compendium of calls to the `CSmoothMaster`:

- `CSmoothMaster()`
  When the object is instantiated, the object will read the information, training and testing data.
- `void ReadTrainingInfo()`
  This reads the training point information to be used for tuning. It is done automatically when creating the `CSmoothMaster` object.
- `void TuneAllY()`
  // Tune all observables, Y.
- `void TuneY(string obsname)`
  Tunes one observable, by observable name.
- `void TuneY(unsigned int iY)`
  Tune one observable, referenced by index.
- `void CalcAllY(CModelParameters *modelpars,vector<double> &Y, vector<double> &SigmaY_emulator)`
  Calculates all observables. CModelParameters object stores information about a single point in model-parameter space. Object described further below.
- `void CalcY(unsigned int iY,CModelParameters *modelpars,double &Y, double &SigmaY_emulator)`
  Calculates observable referenced by index.
- `void CalcY(string obsname,CModelParameters *modelpars,double &Y, double &SigmaY_emulator)`
  Calculates observable referenced by observable name.
- `void CalcAllYdYdTheta(CModelParameters *modelpars,vector<double> &Y, vector<double> &SigmaY_emulator,vector<vector<double>> &dYdTheta)`
  Also calculates derivatives w.r.t. $\vec{\theta}$. Useful for some Markov chain searches in parameter space, e.g. Langevin approaches.
- `void CalcYdYdTheta(string obsname,CModelParameters *modelpars,double &Y, double &SigmaY_emulator,vector<double> &dYdTheta)`
  Same, but for one observable referenced by observable name.
- `void CalcYdYdTheta(unsigned int iY,CModelParameters *modelpars,double &Y, double &SigmaY_emulator,vector<double> &dYdTheta)`
  Same, but by index.
- `void CalcAllYOnly(CModelParameters *modelpars,vector<double> &Y)` and `CalcAllYOnly(vector<double> &theta,vector<double> &Y)` calculates the observable but with the uncertainty.
- `double GetUncertainty(string obsname,vector<double> &theta)` and `GetUncertainty(unsigned int iY,vector<double> &theta)` returns the emulator uncertainty only, for a specific observable.
- `void CalcAllLogP()`
  Prints technical information the User may find helpful in evaluating whether the choice of Λ is reasonable. For each observable, it calculates the ratio of the r.m.s. coefficients of rank-two

to those of rank-one. One would roughly expect the ratio to be unity if Λ is appropriate, but from testing the measure is so noisy that it is not useful on a observable-by-observable basis. It also calculates the probability for the optimum coefficient set. This would be maximized for best choices of Λ, but again is highly sensitive to fluctuations. Thus, this information is not recommended for actual tuning Λ.

- `void TestAtTrainingPts()`
  Compares emulator predictions to full model calculations at training points. Observables should match and uncertainties should vanish.
- `void TestAtTrainingPts(string obsname)`
  Same but for a single observable referenced by observable name.
- `void TestAtTrainingPts(unsigned int iY)`
  Same but for a single observable referenced by index.
- `void TestVsFullModel()`
  This is useful for comparing emulator to model evaluated at points not used for training. You can set the emulator parameter `SmoothEmulator_TestingPts` in the same manner as was done for the emulator parameter `SmoothEmulator_TrainingPts`. It will compare the emulator to the results in the run directories specified here. Usually, one wishes to compare at points not used for training. Fore example, if there were 100 run directories, one might set

  ```
  SmoothEmulator_TrainingPts 0-89
  SmoothEmulator_TestingPts 90-99
  ```

  to train the emulator with the first 90 points and test it at the last 10 points. Tests for each observable are stored in a directory `fullmodel_testdata/`.
- `void WriteCoefficientsAllY()`
  Writes the Taylor coefficients for all observables. Because tuning is so fast, the User can usually avoid reading and writing coefficients, and instead simply re-perform the tuning.
- `void WriteCoefficients(string obsname)`
  Writes only those for one observable referenced by observable name.
- `void WriteCoefficients(unsigned int iY)`
  Writes only for one observable reference by index.
- `void ReadCoefficientsAllY()`
  Reads all Taylor coefficients from file.
- `void ReadCoefficients(string obsname)`
  Reads Taylor coefficients for one observable referenced by observable name.
- `void ReadCoefficients(unsigned int iY)`
  Reads Taylor coefficients for one observable referenced by observable index.

It might prove useful to view the header file for the `master` class, `$SMOOTH_HOME/software/include/msu_smooth/master.h`.

6.6 Other Potentially Useful *Smooth Emulator Objects*

The four programs described above access the functionality through the `CSmoothMaster` object. However, within the main program one may also wish to apply or access other objects with the *Smooth Emulator* framework. Their functionalities are described here:

- `CparameterMap`
  This object stores the emulator parameters described above. It is a simple inheritization of a map, linking string labels to values of various types. The object can read a parameter list from a file, e.g.

```
CparameterMap parmap;
parmap.ReadParsFromFile
 ↪  ("smooth_data/smooth_parameters/emulator_parameters.txt")
```

  The argument can be either a C++ string or a character array. The CSmoothMaster constructor takes a pointer to a CparameterMap object as a argument. If one wishes to print the parameters, the function is `parmap.PrintPars()`. To set a parameter within a program, `parmap.set(string,value)`, where `value` can be any of several types, e.g. bool, double, an integer, long long integer, string, $\cdots$. To retrieve a value from the map, the commands are `getB, getI, getLongI, getS, getD`, $\cdots$, for `bool, int, long long int` $\cdots$ types. For example, one can access options from the `CSmoothMaster` object via

```
double Lambda=master->parmap->getD("SmoothEmulator_LAMBDA",2.5);
```

  If the map did not include `SmoothEmulator_LAMBDA`, it would return a default value of 2.5.

  More detailed information about the `CparameterMap` class is provided in Sec. 9.1.

- `CPriorInfo`
  This object stores information about the prior. The `CSmoothMaster` object includes such an object, and automatically, during its construction, the `CSmoothMaster` object reads in the `smooth_data/Info/prior_info.txt` file and creates the object. The functionalities of potential interest might be addressed from a main program via:

```
smoothmaster.priorinfo.PrintInfo();  // prints out the parameter priors
unsigned int ipar=smoothmaster.priorinfo.GetIPosition(PARAMETER_NAME);
  // finds position given name of parameter (string)
string parname=smoothmaster.priorinfo.GetName(I);
  // finds name given position  I (unsigned int)
```

- `CModelParameters`
  This object stores the information describing a single point in the model-parameter space. It has two vectors storing the true $\vec{X}$ and the scaled $\vec{\theta}$ parameters. As a static variable, it stores a pointer to a `CPriorInfo` object so that it can translate back and forth from $\vec{X}$ to $\vec{\theta}$. The functionalities are fairly easily seen from the definition of its members header file. The ones most likely to be accessed by the User are:

```
vector<double> X;   // model parameters
vector<double> Theta; // scaled model parameters
void TranslateTheta_to_X();  // Given Theta, fill out X
void TranslateX_to_Theta(); // Given X, fill out Theta
void Print();    // Prints model parameters
```

```
void Write(string filename);    // Writes model parameters
void Copy(CModelParameters *mp);  // Copies from another object
void SetTheta(vector<double> &theta);  // Sets model parameters from vector
void SetX(vector<double> &X);  // Sets model parameters from vector
```

The emulator software stores all model-parameter information in these objects. There is a `CTrainingInfo` object within `CSmoothMaster` that stores a vector of such objects.

- `CObservableInfo`
  This object stores general information about all the observables, including their experimental value and experimental uncertainty. The object does not store observable information as calculated by the emulator. These objects are also fairly self-explanatory and the functionality can be ascertained by looking at some of the lines in the header file.

```
unsigned int NObservables;
vector<string> observable_name;
vector<double> SigmaA0;
  // initial guess for spread of coefficients (only used for MC tuning
  ↪ methods)
map<string,unsigned int> name_map;
unsigned int GetIPosition(string obsname);
  // finds position given name of observable
string GetName(unsigned int iposition);
  // finds name give position
void ReadObservableInfo(string filename);
  // reads information about observable, either from
  //smooth_data/Info/observable_info.txt or smooth_data/Info/pca_info.txt
void ReadExperimentalInfo(string filename);
  // reads experimental measurement and uncertainty (used in MCMC)
vector<double> YExp,SigmaExp;
  // experimental measurement information
void PrintInfo();
```

There is such an object in the `CSmoothMaster` class. For example, to print the information about the observables from a main program, one would include the line:

```
 master->observableinfo->PrintInfo();
```

7 Markov-Chain Monte Carlo Generation of the Posterior

7.1 Overview

The `${SMOOTH_HOME}/software/MainPrograms/` directory contains the source code for the main program, `smoothy_mcmc_main.cc`. When compiled it creates an executable, `${SMOOTH_HOME}/software/bin/smoothy_mcmc`, which then performs an MCMC analysis. This program will invoke the emulator in its exploration of the model-parameter space. It will write the trace to file and perform and record some analysis of the trace the User might find useful. Even if the User chooses a different MCMC approach then the one provided with the *Smooth Emulator* software, viewing the source code might prove useful in demonstrating how to invoke the emulator from within a C++ code.

7.2 Experimental Measurement Information

Once the emulator is tuned and before it is applied to a Markov Chain investigation of the likelihood, the software needs to know the experimental measurement and uncertainty. That information must be entered in the `smooth_data/Info/experimental_info.txt` file. The file should have the format:

```
observable_name_1  value_1    exp_uncertainty_1   theory_uncertainty_1
observable_name_2  value_2    exp_uncertainty_2   theory_uncertainty_2
observable_name_3  value_3    exp_uncertainty_3   theory_uncertainty_3
observable_name_4  value_4    exp_uncertainty_4   theory_uncertainty_4
.
```

The second column provides the measured value and the third gives the experimentally reported uncertainty. The fourth column lists the uncertainty inherent to the theory, due to missing physics or to some shortcoming of the model. For example, even if a model has all the parameters set to the exact value, e.g. some parameter of the standard model, the full-model can't be expected to exactly reproduce a correct measurement given that some physics is likely missing from the full model. This should not include any random point-by-point uncertainty of the theory – that is already included through the `ALPHA` option in the emulator uncertainty. For the MCMC software, the relevant uncertainty incorporates both of these, plus the emulator uncertainty. Only the combination of all three, added in quadrature, affects the outcome. We emphasize that this last file is not needed to train and tune the emulator. It is only needed once one performs the MCMC search of parameter space.

7.3 Accounting for Uncertainties

The log-likelihood, LL, for the MCMC generation is assumed to be of a simple form. Summing over the observables I,

$$\begin{aligned} \sigma^2_{I,\mathrm{tot}} &= \sigma^2_{I,\mathrm{exp}} + \sigma^2_{I,\mathrm{theory}} + \sigma^2_{I,\mathrm{emulator}}, \\ LL &= -\sum_I \frac{(Y_{I,\mathrm{exp}} - Y_{I,\mathrm{emu}})^2}{2\sigma^2_{I,\mathrm{tot}}} - \ln(\sigma_{I,\mathrm{tot}}). \end{aligned}$$

The likelihood behaves as e^{LL}. This form presumes that the uncertainties are Gaussian and that the uncertainties for the various observables are independent. The last term above derives from the normalization factor for the Gaussian. One can view smoothdraft.pdf for more details.

Correlated errors are not explicitly addressed in *Smooth Emulator* software. However, one can always add additional model parameters that describe contributions to multiple observables due to some unknown factors, such as an experimental efficiency or some neglected physics. This can be a convenient and sound perspective from which one can incorporate both experimental and theoretical systematic uncertainties. Sometimes such parameters are called "nuisance" parameters, because they usually describe effects that are not of direct interest to the reader, but do impact the degree to which one might constrain the parameters of greater interest. However, the moniker "nuisance" can be rather unfortunate because although some parameter might be a nuisance to one scientist, e.g. the efficiency of the detector, that same parameter might represent the life's work on another scientist, who might find the constraint of such a parameter of great interest. Nuisance and non-nuisance parameters are treated on equal footings.

7.4 MCMC Options

Next, one needs to edit the options file `${MY_ANALYSIS}/smooth_data/Options/mcmc_options.txt`. An example file is:

```
# This is for the MCMC search of parameter space
# (not for the emulator tuning)
#LogFileName smoothlog.txt # comment out for interactive running
# If success rate approaches zero, reduce MCMC_METROPOLIS_STEPSIZE, if it
↪  approaches 1.0, increase it
MCMC_METROPOLIS_STEPSIZE 0.06
MCMC_IGNORE_EMULATOR_ERROR false
MCMC_RANSEED  12345
MCMC_NBURN 5
MCMC_NSKIP 10000
MCMC_NTRACE 100000
```

As was the case with *Smooth Emulator*, each option has a default value. The `CMCMC` class has a parameter map object. This map stores the options, and uses them to set the corresponding data in the `CMCMC` object. The values of `MCMC_NBURN`, `MCMC_NTRACE` and `MCMC_NSKIP` are stored by the `CMCMC` parameter map, but do not correspond to any members of the class. When the User wishes to set the number of trace points in the main program, they do so by changing the arguments to the `PerformTrace(..)` function. The User can set these arguments through the parameter map functionality, which can provide a convenient way to set the arguments through the Options file. See Chapter 7.6 below for an example of how this is used.The options are described below.

7.4.1 `LogFileName (if commented out will write to screen)`

To run interactively, leave this line commented out. Otherwise, output will be directed to the designated file.

7.4.2 MCMC_METROPOLIS_STEPSIZE (default 0.05)

Metropolis algorithms require taking random steps in $\vec{\theta}$ space. If the steps are small, it takes longer to explore the space, but if the steps are very long, the success rate of the Metropolis steps becomes low. Maximum efficiency occurs when the Metropolis success rate, which is provided during running, is near 50%. Rates of 20% or 80% are also fine, but if the rate is only a few percent, the User should reduce the parameter, and if the success rate becomes close to 100%, the User should increase the parameter.

7.4.3 MCMC_IGNORE_EMULATOR_ERROR (default false)

If this flag is set to `true` the emulator uncertainty will be ignored. This significantly increases the speed of the MCMC procedure, but should not be done if the emulator error is significant.

7.4.4 MCMC_RANSEED (default 12345)

Sets seed for random number generator.

7.4.5 MCMC_NBURN, MCMC_NTRACE, MCMC_NSKIP (no defaults)

These are different than the other options in that they do correspond to any member of the `CMCMC` class. Instead, they are simply stored by `CparameterMap` object of the `CMCMC` class for convenience. The variables describe the number of random steps to be taken during the burn-in stage (where the trace is not being recorded), the number of steps in the trace to be recorded once the burn-in stage has ended, and the number of steps between recorded steps. Because successive steps are highly correlated it makes sense to not record every step. For example, if the Metropolis success rate is 50%, then half the points are identical to the previous point.

The relevant variables are set when the CMCMC object calls the function `PerformTrace(int Ntrace,int Nskip)` in the main program. When writing a main program, the User can set `Ntrace` or `Nskip` through the parameter map, which is a member of the `CMCMC class`. This is illustrated below in Chapter 7.6, where the source code for the main MCMC program is described. The User can always simply call the `PerformTrace` function with whatever arguments they wish and never use the options stored in the parameter map. The parameter map functionality is described in Sec. 9.1.

7.5 Running the MCMC Program

To run the provided MCMC program, move to the analysis directory, and run the program `mcmc`. Output should look something like this:

```
${MY_ANALYSIS}% ${SMOOTH_HOME}/bin/smoothy_mcmc
At beginning of Trace, LL=-68.764478
At end of trace, best LL=1.563806
Best Theta=
```

```
0.249554  0.153237  0.190531  0.210907  0.058929  0.230998
Metropolis success percentage=54.090000
finished burn in
At beginning of Trace, LL=-5.477040
finished 10%
finished 20%
finished 30%
finished 40%
finished 50%
finished 60%
finished 70%
finished 80%
finished 90%
finished 100%
At end of trace, best LL=1.678685
Best Theta=
0.269108  0.099867  0.185094  0.204939  0.037417  0.207398
Metropolis success percentage=53.609600
writing, ntrace = 100001
writing, ntrace = 100001
```

For the sake of numerical efficiency ‘‘`Metropolis success percentage`’’ should be in the range of 50%. If the efficiency is very near 100%, it suggests the step sizes might be too small to most efficiently explore the entire model-parameter space. If the efficiency is near zero, the step size might be too large and too few successful Metropolis steps will be successful. This affects only the efficiency, not the validity, so any success percentage between 10% and 90% should suffice. In the output, ‘‘`LL`’’ refers to the log-likelihood. At the end of the burn-in, one hopes that best value of `LL` is not much lower than the best value found from the entire trace. If not, one should probably increase the value of `MCMC_NBURN`. The values of ‘‘`Best Theta`’’ refer to the point in the trace with the highest `LL`.

Information about the trace is found in the files located in the directory `smooth_data/mcmc_trace/`:

- `trace_theta.txt`: A list of the scaled model-parameter values, $\vec{\theta}$, from the posterior sampling.
- `trace_X.txt`: A list of the non-scaled model-parameter values, $\vec{X}$, from the posterior sampling.
- `xbar_thetabar.txt`: The average of the parameter values ($\vec{X}$), and the scaled values ($\vec{\theta}$) from the posterior.
- `CovThetaTheta.txt`: This gives the $N_{\rm par} \times N_{\rm par}$ covariance matrix $\langle \delta\theta_i \delta\theta_j \langle$, describing the size and shape of the points in the trace.
- `CovThetaTheta_eigenvalues.txt`: That eigenvalues of that matrix
- `CovThetaTheta_eigenvecs.txt`: The eigenvectors
- `ResolvingPower.txt`: An $N_{\rm pars} \times N_{\rm obs}$ matrix describing the influence of each observable in resolving each model parameter.

7.6 Reviewing the MCMC Source Code

Finally, we review the source code in ${SMOOTH_HOME}/software/MainPrograms/mcmc_main.cc:

```
int main(){
   NBandSmooth::CSmoothMaster master;
   master.TuneAllY();
   NBandSmooth::CMCMC mcmc(&master);
   unsigned int Nburn=mcmc.parmap->getI("MCMC_NBURN",1000);  // Steps for burn in
   unsigned int Ntrace=mcmc.parmap->getI("MCMC_NTRACE",1000); // Record this many
    ↪  points
   unsigned int Nskip=mcmc.parmap->getI("MCMC_NSKIP",5); // Only record every
    ↪  Nskip^th point

   mcmc.PerformTrace(1,Nburn);
   CLog::Info("finished burn in\n");

   mcmc.PruneTrace(); // Throws away all but last point
   mcmc.PerformTrace(Ntrace,Nskip);
   mcmc.WriteTrace(); // Writes trace
   mcmc.EvaluateTrace();

   return 0;
}
```

This is mostly self-explanatory. If one wishes to avoid writing out the trace, the line `mcmc.WriteTrace` can be deleted. If the User wishes to run the code in batch mode, the output can be directed to a file, `mcmc_log.txt`, by adding the line

```
LogFileName mcmc_log.txt
```

to the options file `smooth_data/Options/mcmc_options.txt`. The line `mcmc.EvaluateTrace()` will evaluate the trace and calculate the resolving power and covariances.

If one has previously trained the emulator, one can use the known value of Λ rather than recalculating it. Normally, this saves very little time, but might be noticeable when the number of model parameters is large.

7.7 The MCMC header file

In the main repository one can find a header file, ${SMOOTH_HOME}/software/inlude/msu_smooth/mcmc.h. Perusing this file is a good way to understand the provided functionality of the CMCMC class.

```
class CMCMC{
   public:
      bool IGNORE_EMULATOR_ERROR;
      CparameterMap *parmap;
      CSmoothMaster *master;
```

```
    Crandy *randy;

    CMCMC();
    CMCMC(CSmoothMaster *master);
    unsigned int NPars,NObs;
    vector<vector<double>> trace;
    string trace_filename,Xtrace_filename;
    double stepsize;

    void ClearTrace(); // erases trace info so one can start over, resets at
    ↪  theta=0.
    void PruneTrace(); // erases trace, except for last point

    void PerformTrace(unsigned int Ntrace,unsigned int Nskip);
    void PerformMetropolisTrace(unsigned int Ntrace,unsigned int Nskip);
    void WriteTrace();
    void ReadTrace();
    void EvaluateTrace();

    Eigen::VectorXcd stepvec,stepvecprime,dTdTEigenVals;
    Eigen::MatrixXd dThetadTheta;
    Eigen::MatrixXcd dTdTEigenVecs;

    void CalcLL(vector<double> &theta,double &LL);
    CLLCalcSmooth *llcalc;
    static CPriorInfo *priorinfo;
};
```

The functionalities that are most likely to be incorporated are the functions `ClearTrace()`, `PruneTrace()`, `PerformTrace()`, `WriteTrace()`, `EvaluateTrace()` and `CalcLL()`. These functions are mostly self-explanatory. The function `WriteTrace()` will write the two files with trace information described in Sec. 7.5. The function `CalcLL(vector<double> &theta,double &LL)` calculates the posterior log-likelihood, `LL`, for the point $\vec{\theta}$.

The function `EvaluateTrace()` considers the trace, assuming it has been calculated. After evaluating the trace it finds and writes the averages and covariances mentioned in Sec. 7.5. It also writes the file describing the resolving power for each observable w.r.t. the constraint of each observable. All the trace-evaluation output files are written to files in the directo`rsmooth_data/MCMC/` directory.

8 Plotting Programs

8.1 Overview

The *Smooth Emulator* distribution includes three plotting programs, all based on Pythons's Matplotlib. The first program allows the User to visualize the accuracy of the emulator by comparing emulator predictions to full-model calculations at random points not used to train the emulator. The other two plotting programs illustrate the results of the MCMC constraint of model-parameter space.

8.2 Preparing the Data Files for the Plots

The plotting programs are contained the `${MY_ANALYSIS}/figs/` directory. The three plotting programs are located in the three subdirectories, `figs/YvsY`, `figs/posterior` and `figs/resolvingpower`. Before running the plotting programs, one must first create several data files and move them to the `figs/figdata` directory. This directory will hold all data required to make the plots. Thus, none of the plotting functionality depends on the location of the `figs/` directory, as long as the `figdata/` directory is a subdirectory of the same directory as `YvsY/`, `posterior/` and `resolvingpower/`.

The first data file is `figdata/prior_info.txt`. This file provides the plotting programs with the names of the model parameters, both the short-hand names and the LaTeXname. To create the file, one can first copy the file of the same name from the `smooth_data/Info/` directory, then edit. The file should then be edited to have three columns describing the model parameters:

```
parameter_name_1 prior_type_1 axis_name_1
parameter_name_2 prior_type_2 axis_name_2
 .
```

The first two columns are identical to those in `smooth_data/Options/prior_info.txt`. For example, the model parameter might be named `initial_temperature`. The second column is the prior type, either `uniform` or `gaussian`. The third column, which is new, provides the strings for labeling the axes in the plots. For example, this might be `$T_0$`. These are not purely LaTeXformats, but are those used by Matplotlib, which differ from standard LaTeXbecause some of the backslash commands require double backslashes, e.g. `\r` needs to be `\\r`. The names in the last column are arbitrary and are only used to label axes. All elements of the file must be strings of characters without spaces.

The second file, `observable_info.txt`, is also created by initially copying the file of the same name in the `smooth_data/Info/`. The new file, `figdata/observable_info.txt` must then be edited. As the name implies it provides the names of the various observables to be plotted. The file format is:

```
observable_name_0  axis_name_0
observable_name_1  axis_name_1
 .
```

The first column is exactly the same as the original file, and the last column provides the LaTeX-like labels for the axes.

If the User wishes to plot the comparison between emulator predictions and actual full-model values, they must move the directory `smooth_data/FullModelTestingRuns/` to the `figs/figdata` directory.

```
% cp -r $\{MY_ANALYSIS\}/smooth_data/FullModelTestingRuns
↪  $\{MY_ANALYSIS\}/figs/figdata/
```

In the tutorial, that directory, and its files, are created by running the `fakefullmodel` program, which calculated the full-model values at a set of randomly-chosen points. Within that directory are separate files providing the comparison between the full-model and the emulator predictions. The files are named `YvsY_[observable-name].txt`. Here `[observable-name]` must match the ascii names from `figdata/observable_info.txt`. If the calculation compares observables at n points, the file format for each of the $N_{\rm obs}$ files is:

```
Y1_full   Y1_emulator emulator_uncertainty_1  accuracy_1
Y2_full   Y2_emulator emulator_uncertainty_2  accuracy_2
  .
Yn_full   Yn_emulator emulator_uncertainty_n  accuracy_n
```

The first and second columns give the full-model and emulator values for each of the randomly chosen points in model-parameter space. The third column lists the emulator's prediction of its uncertainty at those points. The final column lists the deviation between the full-model and emulator values scaled by the uncertainty. Only the first three columns are used by the plotting program.

If the User wishes to visualize the posterior by sampling the MCMC trace or if they wish to plot the resolving power extracted from the MCMC trace data, they should move the following files to the figs/figdata/ directory.

```
% cp $\{MY_ANALYSIS\}/smooth_data/MCMC/trace_theta.txt
↪  $\{MY_ANALYSIS\}/figs/figdata/
% cp $\{MY_ANALYSIS\}/smooth_data/MCMC/ResolvingPower.txt
↪  $\{MY_ANALYSIS\}/figs/figdata/
```

8.3 Running `YvsY.py` to Compare Emulator Predictions to the Full-Model

To perform this comparison, one must enter the `figs/YvsY/` directory and run the Python script:

```
 ${MY_ANALYSIS/figs/YvsY% Python3 YvsY.py
--- Observables ---
iY= 0   observable_name_0
iY= 1   observable_name_1
iY= 2   observable_name_2
iY= 3   observable_name_3
 .
Enter iY to designate observable: XXX
XXX of 100  points within 1 sigma
```

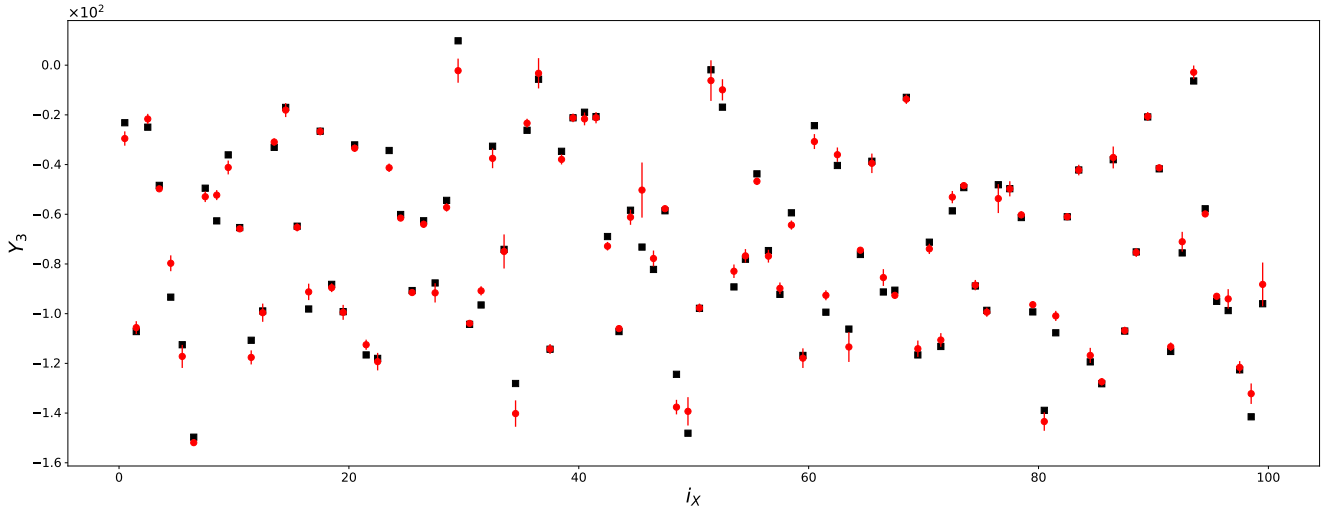


Figure 8.1: For the case plotted here 100 model-parameter points, noted in the plot by i_x, were evaluated. The black squares show the full-model values and the red points show the emulator predictions along with their error. When the program finishes the plot is also prints out how many of the comparisons were within one unit of the emulator uncertainty, denoted by `XXX` in the output above. If the emulator were to accurately state its uncertainty, 68.3% of the points would be within one σ.

As shown above, the script prompts the User to name which observable is being considered. The User should enter $n =$`'0','1','2'`$\cdots$ to denote the observable in the provided order. If the User enters '3', emulator predictions for the observable `observable_name_3` will be evaluated. A plot should appear:

If the User wishes to view the PDF file of the plots, rather than the Python version, they can go into the file and comment out the `plt.show` line, then uncomment the lines calling the system commands:

```
 .
# if you have Mac OS and want to see pdf file, comment out previous two lines and
 ↪  uncomment line below
#os.system("open -a Preview "+outputfilename);
# if you have Linux and want to see pdf file, comment out previous two lines and
 ↪  uncomment line below
#os.system("okular "+outputfilename+"&");
 .
```

It is likely the User will wish to alter the plots for their purpose. Hopefully, the Python script is sufficiently straight-forward to make such edits without too much difficulty.

8.4 Running `posterior.py` to View the Posterior Constraints of the Model-Parameter Space

To view the posterior distribution, enter the `figs/posterior/` directory and run the Python script:

```
 ${MY_ANALYSIS/figs/YvsY% Python3 posterior.py
Number of points in trace = XXX
```

The script reads the MCMC trace data and creates a plot, an example of which is shown here.

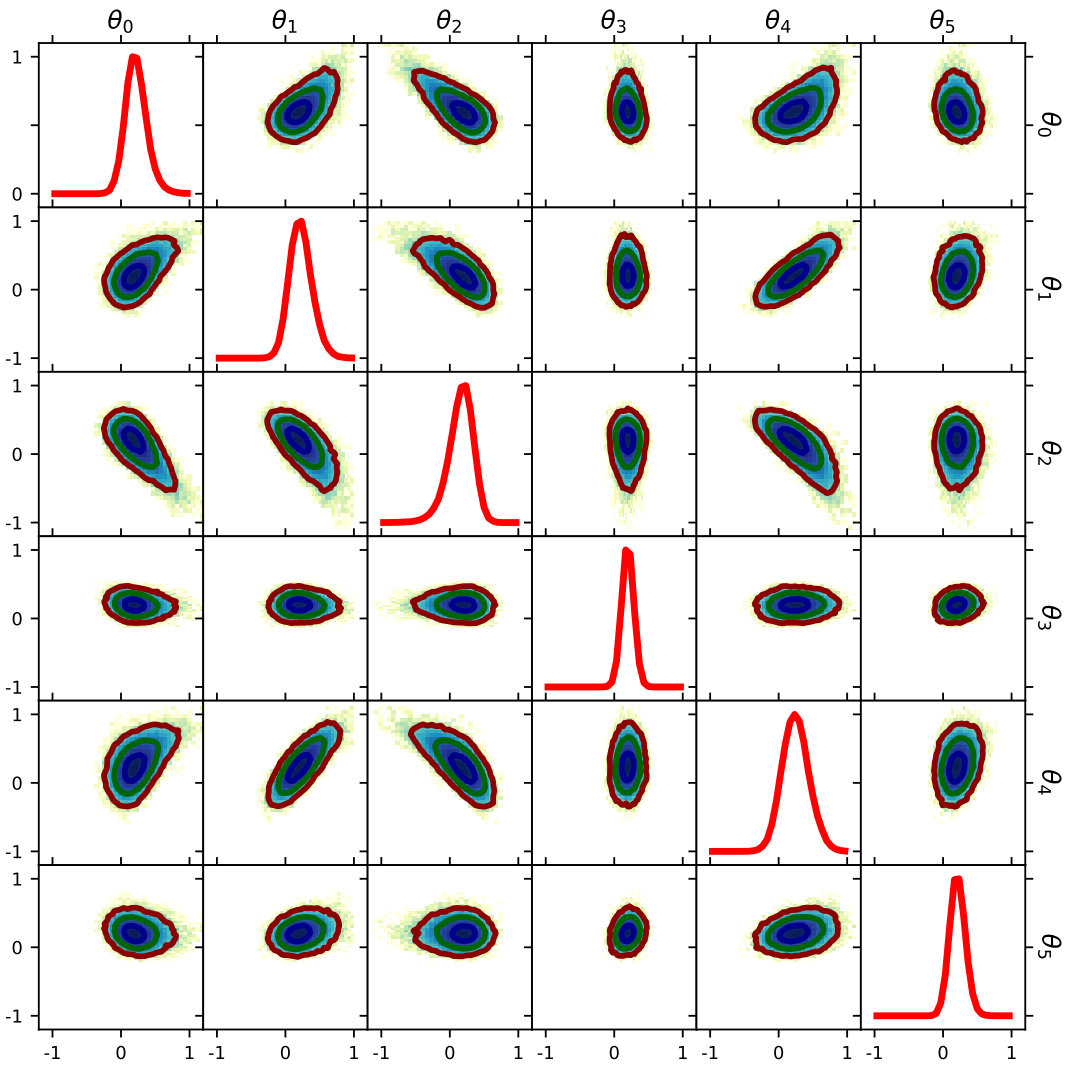


Figure 8.2: As was the case before one can edit the lines near the bottom of the file to display the PDF file.

For this example, the "experimental" data was generated by running the model with each model parameter set to $\theta_i = 0.2$. One can see that the procedure was rather successful in finding the correct values, which serves as a self-consistency check.

The User should note that the scaled coordinates, $\vec{\theta}$, are used instead of the real coordinates. This choice was made simply because the unscaled coordinates might have complicated bounds, which might be difficult to display given the small sub-panels. The User should also note that the posterior includes the constraints of the prior. Hence, if some parameter had a Gaussian prior, and was not at all further constrained by the analysis, it would still appear as a non-uniform distribution in the plot.

The User needs to choose what subset of the observables they wish to consider in the plot. This is done by editing the Python script, `figs/posterior/posterior.py`. One must edit the line

```
 .
ParsToPlot=[0,1,2,3,4,5]
 .
```

If there were 6 observables, this would then consider all of them. Otherwise, one should adjust the list accordingly.

8.5 Running `RP.py` to View the Resolving Power

One measure of an observable's resolving power with respect to constraining a specific model parameter is defined as the amount the posterior average of the constrained model parameter, $\langle\langle X\_i\rangle\rangle$, is likely to change given a change of the observable, $\langle\langle Y\_i\rangle\rangle$, scaled by the variance of Y_i in the prior. The MCMC analysis program created a file `smooth_data/MCMC/ResolvingPower.txt`, which the User has moved to the `figs/figdata/` directory. By entering the `figs/resolvingpower/` directory and running the Python script, one creates a figure showing the resolving power,

```
${MY_ANALYSIS/figs/YvsY% Python3 RP.py
NPars= XXX NObs= XXX
```

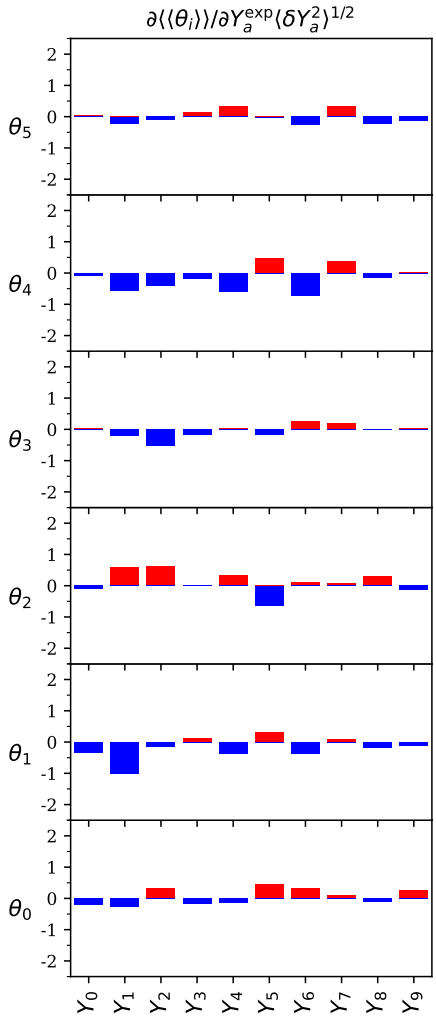


Figure 8.3: An example of the resolving power. Positive/negative values mean that the extracted model parameters are increased/reduced if the observable grows. If the magnitude of the bar is large, the extracted model parameter is strongly sensitive to the observable.

9 Utility Classes

Smooth Emulator software employs several "utility" classes. These can be used for a variety of purposes, and might at times be invoked in the source code for the main programs. Thus, the User might find further explanation of these provided functionalities. Here, we present the functionalities of three such classes. The relevant header files for these classes can be found in `${SMOOTH_HOME}/software/include/msu_smoothutils/`. The first is the `CparameterMap` class. This is basically an extension of a C++ map, with some added functionality such as reading in the key/value pairs from files and to set elements with default values. These objects are used to read in the options from text files. The second class is the `CLog` class. This is used for directing output to allow the User to toggle between sending output, including error messages, to a specified file rather than interactively (to the screen). The third class described here is `Crandy`. This simplifies calling random number generators. Even though the User might not be writing source code where random numbers are employed, the User may wish to set the seeds for the `Crandy` random number generators.

9.1 The `CparameterMap` Class

It is common that the user will wish to either extract or set parameter that correspond to the various options inherent to those classes. This is done through an object in each of these classes that stores such parameters. These objects are defined by the `CparameterMap` class, and as the name implies, this class provides map-like functionality.

For example, if the User wishes to write programs that change options during the running, they can use the parameterMap class, which has an object within the `CTPO` class, `tpo->parmap`. For example, the User could set `LAMBDA=3.0` by adding the following line to the main program:

```
 .
tpo->parmap.set("TPO_LAMBDA",3.0);
 .
```

This creates a key-value pair, where the key is the string `TPO_LAMBDA` and the value is 3.0. If the User wanted to have the program report the current value, they could add a line, e.g.,

```
 .
int NMC=tpo->parmap.getI("TPO_NMC",0);
 .
```

Here the first argument is the key. The second argument is the default value, which will be used if the map does not include the given string. There are similar objects in the `MCMC` class. The User may have created one of these objects, e.g.,

```
      NBandSmooth::CTPO *tpo=new CTPO();
      NBandSmooth::CSmoothMaster smoothmaster;
      NBandSmooth::CMCMC mcmc(&smoothmaster);
```

All of these three classes have a `CparameterMap` object. For each class the relevant options files are read and the parameter maps are populated during the instantiation. The User can then set or access members of the class through the objects `parmap` member. For example:

```
        double Lambda=smoothmaster.parmap->getD("SmoothEmulator_LAMBDA",2.5);
        double stepsize=mcmc.parmap.getD("MCMC_METROPOLIS_STEPSIZE",0.06);
```

The parameterMap objects were originally populated by reading the options files, smooth_data/Options/*options.txt. Thus, the options can be accessed through the names given in those files.

The parameterMap class is basically an extension of C++ maps. The header file, ${SMOOTH_HOME}/software/include/msu_smoothutils/parametermap.h, is:

```
class CparameterMap : public map<string,string>{
public:
  bool getB(string ,bool);
  int getI(string ,int);
  long long int getLongI(string ,long long int);
  string getS(string ,string);
  double getD(string ,double);
  vector< double > getV(string, string);
  vector< string > getVS(string, string);
  vector< vector< double > > getM(string);
  vector< vector< double > > getM(string, double);
  void set(string, double);
  void set(string, int);
  void set(string, unsigned int);
  void set(string, long long int);
  void set(string, bool);
  void set(string, string);
  void set(string, char*);
  void set(string, vector< double >);
  void set(string, vector< string >);
  void set(string, vector< vector< double > >);
  void ReadParsFromFile(const char *filename);
  void ReadParsFromFile(string filename);
  void PrintPars();
  char *message;
}
```

The first argument of most of the "set" methods specifies the key in the map, and the second is the value to be set. For the "get" methods, the first value is the key and the second is the value to be used should the key not exist in the map. The methods getB, getI, getLongI, getS and getD return booleans, integers, strings or doubles. The software does have the capability to set and find vectors, but that shouldn't be necessary in this instance. All the options listed for Smooth Emulator software are set using this functionality.

9.2 The CLog Class

This object has only static members, so there is only a single CLog object in a program. If CLog::INTERACTIVE is set to true (the default) anything the program writes through the CLog

object will be sent to the screen. Otherwise, the output will be sent to the file defined by the string `CLog::logfilename`. There are three basic functions, `CLog::Init(string filename)`, which sets the output filename (if not interactive), `CLog::Info(string message)` and `CLog::Fatal(string message)`. For example if one writes the program:

```
#include "msu_smoothutils/commonutils.h"
using namespace NBandSmooth;
int main{
        double x=3;
        CLog::Info("x="+to_string(3));
        return 0;
}
```

the program will write `x=3` to the screen. As another example, the program

```
#include "msu_smoothutils/commonutils.h"
using namespace NBandSmooth;
int main{
  CLog::Init("myoutput.txt");
  double x=3;
  CLog::Info("x="+to_string(3));
  return 0;
}
```

will write the same output to the file `myoutput.txt`. If the `Info()` function is replaced by `Fatal()` the same behavior will occur, but writing the statement will be followed by the program exiting (according to `exit(1)`).

The class definition from the header file, `${SMOOTH_HOME}/software/include/msu_smoothutils/log.h` is

```
class CLog{
  public:
    static bool INTERACTIVE;
    static string logfilename;
    static FILE *fptr;
    static void Init(char *logfilename_in);
    static void Init(string &logfilename_in);
    static void Info(string message);
    static void Fatal(string message);
    static const int CHARLENGTH=300; // ignore if using only strings
};
```

Single messages should be less than 300 characters.

9.3 The `Crandy` Class

As the name suggests this object assists with the generation of random numbers with various distributions. A simple example of the class is:

```
#include "msu_smoothutils/commonutils.h"
using namespace NBandSmooth;
int main{
  Crandy *randy=new Crandy(12345);
  double x=randy->ran();
  printf("x=%g\n",x);
  return 0;
}
```

The program prints out a number between 0 and 1. One can reset the seed via `randy->reset(int seed)`. The class defined in the header file, `${SMOOTH_HOME}/software/include/msu_smoothutils/randy.h`, is

```
class Crandy{
public:
  Crandy(int iseed);
  int seed;
  double threshold,netprob;
  double ran();
  double ran_gauss(); // return x propto exp(-x^2/2), <x^2> = 1
  void ran_gauss2(double &g1,double &g2);
  double ran_exp(); // return x propto exp(-x)
  double ran_lorentzian(); // return x propto 1/(1+4x^2), i.e. multiply by full
  ↪  width
  double ran_invcosh();    // return x propto 1/cosh(x), <x^2> = PI^2/4.
  void reset(int iseed);
  int poisson();
  void set_mean(double mu);  // For Poisson Dist
  void increase_threshold();
  void increment_netprob(double delN);
  bool test_threshold(double delprob);
  //std::mt19937 mt;   // 32 bit
  std::mt19937_64 mt;   // 64 bit
  std::uniform_real_distribution<double> ranu;
  std::normal_distribution<double> rang;
  std::poisson_distribution<int> ranp;
};
```

Some of these functions are self-explanatory. For the purpose of emulation, the User is most likely to wish to reset the seeds.

In *Smooth Emulator* software the classes `CTPO` (for training point optimization) and `CMCMC` (for MCMC) have their own `Crandy` pointers. The User can then access the random number functionality in such cases with lines such as

```
int main{
  .
  CTPO tpo;
  .
```

```
  double x=tpo.randy->ran();
  .
}
```

10 Tutorial

10.1 Overview

Working through this section constitutes a tutorial, consisting of the following steps.

1. Copy the required files from the template directory to your space, and compile the main programs.
2. Set up the fake information files describing the priors, observable names and experimental measurements.
3. Run `trainingpoint_optimizer` to generate the model-parameter values at which the full model will be trained.
4. Run a fake (for the purposes of this tutorial) full model to generate the observables for each of the full-model runs. This includes files both for tuning the emulator and also for testing the emulator.
5. Run a program, `smoothy_testattrainingpts`, which builds and trains an emulator and checks that it reproduces the training points.
6. Run a program, `smoothy_testvsfullmodel`, which creates data files comparing emulator predictions to full-model runs at random points not used to train the emulator. Create a plot illustrating the comparison.
7. Run an MCMC program, `smoothy_mcmc`, that produces a trace through model-parameter space that consistently represents the posterior probability by comparing emulator predictions to fake experimental data. Run Python plotting programs that read MCMC data and provide visual representations of the model-parameter-space posterior and of the observable-parameter resolving power.

The text here includes a much-condensed version of what appears in previous sections of the manual.

10.2 Setting up the Directory and Compiling the Main Programs

First download the repository. If you wish to download it into your home directory, `/Users/CarlosSmith/`, they need to open a command-line window and enter:

```
/Users/CarlosSmith% git clone https://github.com/bandframework/bandframework.git
```

The command line examples in this section assume that the prompt is writing down the current path followed by a percent sign.

After downloading the repository, copy the files to convenient places. An analysis directory template is provided with the intention that you will copy the directory to your own space, then use this as a foundation from which to embark on your own analysis. This template also provides the files for a tutorial. Assuming you downloaded the repository to /Users/CarlosSmith/bandframework, the following commands copy the needed files to some more convenient locations.

```
..% cp -r /Users/CarlosSmith/bandframework/software/SmoothEmulator
↪  /Users/CarlosSmith/SmoothHome
..% cp -r ${SMOOTH_HOME}/AnalysisTemplate /Users/CarlosSmith/MyAnalysis
```

Here, `../SmoothHome`, `../MyAnalysis` can be replaced by any directory name you choose. For the rest of the tutorial the following shorthand will be applied

`${SMOOTH_HOME}`	Location of the *Smooth Emulator* portion of the git Repository, e.g. the directory tree copied from `/Users/CarlosSmith/bandframework/software/SmoothEmulator`
`${MY_ANALYSIS}`	Directory tree copied from `${SMOOTH_HOME}/AnalysisTemplate`. Work spaces where parameter files, data, results and figures are created and stored. You may have several different such directories

You should be sure to have a C++ compiler that handles the C++-20 standard. You should also have Cmake and the Eigen linear algebra packages installed. You should also need a Python distribution installed along with the matplotlib package. You can now compile the main libraries and executables

```
..% cd ${SMOOTH_HOME}/software
${SMOOTH_HOME}/software% cmake .
${SMOOTH_HOME}/software% make
```

The repository was organized to encourage you to edit any files in `${SMOOTH_HOME}/software/MainPrograms`.

Several executables should now appear in `${SMOOTH_HOME}/bin/`: `trainingpoint_optimizer`, `smoothy_testattrainingpts`, `smoothy_testvsfullmodel`, `smoothy_calcobs`, `smoothy_mcm`, which all involve the emulator. Two other executables, `fakefullmodel` and `fakeinfo` are only used for the tutorial. The reason these are compiled in a space separate from the main libraries, is that you may well wish to create your own main programs. This arrangement allows you to compile your own versions, while leaving the bulk of the source code unchanged.

You need to define some environmental variables for your shell.

```
% export SMOOTH_HOME=${SMOOTH_HOME}
```

Here, `${SMOOTH_HOME}` is the full path to the location chosen above. You might find it convenient to add `${SMOOTH_HOME}/bin` to your path.

```
..% export PATH=${PATH}:${SMOOTH_HOME}/bin
```

These last three `export` commands can be placed in your shell initialization script, e.g. `/Users/CarlosSmith/.zshrc` or `/Users/CarlosSmith/.bashrc`. Otherwise you would need to redefine the shell variables each time you log on.

10.3 Creating the Necessary Info Files

You will run software from the `${MY_ANALYSIS}/` directory. Before you can run the *Smooth Emulator* executables they must create text files that describe the model-parameter priors and list

the observable names. These three files need to be in the ${MY_ANALYSIS}/smooth_data/Info/ directory. For your own project, you would likely create these files by hand, but, for the purpose of this tutorial you can create all three files by running the program,

```
..% cd ${MY_ANALYSIS}
${MY_ANALYSIS}% ${SMOOTH_HOME}/bin/fakeinfo
```

The first file, smooth_data/Info/prior_info.txt, describes the model parameters and their priors. This file is For the purposes of this tutorial, we consider a model with six model parameters:

```
# par_name dist_type  xmin/centroid xmax/width  SensitivityScale
   par0 gaussian 0 100    1.00000
   par1 uniform -100 100  1.00000
   par2 gaussian 0 100    1.00000
   par3 uniform -100 100  1.00000
   par4 gaussian 0 100    1.00000
   par5 uniform -100 100  1.00000
```

Thus, the model has six parameters. The second entry in each line is either uniform or gaussian. If the entry is uniform, the last two numbers represent the range of the uniform prior, $x_{\min}$ and $x_{\max}$. If the second entry is gaussian the third entry represents the center of the Gaussian distribution and the fourth represents the width. The final column represents the sensitivity scale, which must be between zero and 1.0. For the most impactful parameters, this should be set to 1.0. If a parameter is expected to provide little variance compared to the most impactful parameters, this should be set to some fraction. Roughly, if a parameter is expected to provide half as much variance as the most important parameters, it should be set to 0.5. For a full model, you would replace this file with one appropriate for your own model. This file is required for training-point optimization, for tuning, and for the MCMC programs.

The second file is smooth_data/Info/observable_info.txt. This describes output values from the model. In the template, the file describes, in this case, ten observables:

```
# ObservableName   ALPHA
   obs0 0
   obs1 0
   obs2 0
   obs3 0
   obs4 0
   obs5 0
   obs6 0
   obs7 0
   obs8 0
   obs9 0
```

The first entry in each line simply provides the names of the observable which will be processed in the Bayesian analysis. The second entry describes the point-to-point noise. Here, ALPHA is the noise as a fraction of the characteristic variation of the observable, σ_A. Noise refers to variations of the model that would occur if the model were rerun with the same model parameters, which is typically due to some finite sampling inherent to the model, e.g. if the particle is calculating the average energy of particles using a simulation, and if a finite number of simulations and particles are

sampled, the observable might vary from run to another. If one expects that random uncertainty to be 5% of the variation of the observable throughout the model parameter space, then `ALPHA` would be 0.05. *Smooth Emulator* software assumes that this noise is the same for all the full calculations of the same observable. I.e., if 20 training runs were performed at different points in model-parameter space, the software assumes that the `ALPHA` was not much different at one training point than at another. If `ALPHA` is set to zero, the emulator will exactly reproduce the observables from the full model when evaluated at the training points. If one performed training runs with simulations using grossly different numbers of events at one training point vs another, then this shortcoming could potentially be an issue. This file is required by both the tuning and MCMC programs.

The third file, `smooth_data/Info/experimental_info.txt`, describes the actual measurements and only comes into play at the time the MCMC is being performed. The file in the tutorial might look like this:

```
obs0  57.840878  10.0 0.0
obs1  -85.613188  10.0 0.0
obs2  43.335184  10.0 0.0
obs3  40.159450  10.0 0.0
obs4  38.013337  10.0 0.0
obs5  64.492673  10.0 0.0
obs6  -195.300157  10.0 0.0
obs7  99.587427  10.0 0.0
obs8  7.963056  10.0 0.0
obs9  68.844147  10.0 0.0
```

The names of the observables must match those listed in the `observable_info.txt` file. The second column is the value reported by the experiment and the third column is the reported uncertainty. The uncertainty cannot be set to zero. The last column represents the model's uncertainty due to missing physics, i.e. not the uncertainty due to noise in the calculation. The contribution from random noise is incorporated into the emulator uncertainty. The uncertainty due to missing physics can be thought of as the error, or deviation from a perfect measurement, one might expect if one used the precisely correct parameters. One can think of this as the systematic error of the theory on which the model is built. For the purpose of the MCMC, the three uncertainties, that of the emulator, that from the experiment, and the model's systematic theoretical error, are all added in quadrature to provide the uncertainty relevant for calculating the log-likelihood in the MCMC.

For a real project, you need to edit the three files, probably typing the information all in by hand.

10.4 Selecting Training Points with `trainingpoint_optimizer`

Before running `trainingpoint_optimizer` you should inspect or edit the options file, `smooth_data/Options/tpo_options.txt`. A template for this file is provided:

```
TPO_Method  MCQuadratic  # can be "MC", "MCSphere", "MCSimplex",
↪  "MCSimplexPlus1", "MCQuadratic", "MCSphereQuadratic"
TPO_NTrainingPts    0   # only relevant for OptimizeMethod="MCSphere" or "MC"
TPO_NMC  100000
TPO_ALPHA 0.01 # does not have to be the same value used to train emulator
```

```
TPO_Include_LAMBDA_Uncertainty true # dangerous to set this to false
TPO_FullModelRunsDirName  smooth_data/FullModelRuns
#TPO_ReadPoints 0,1,4-7,10,11-13,21
#TPO_FreezePoints 0,1,4-7,10,11-13,21
```

Options are described in detail in Chapter 4. For this case `TPO_Method` was set to `MCQuadratic`, which sets the number of training points equal to the minimum number for constraining all the expansion coefficients up to 2nd order, which for 6 model parameters is 28. If you had set `TPO_Method` to `MC`, then the number of training points would have been set by the `TPO_NTrainingPts` option. It is safer to set `TPO_ALPHA` to non-zero. Even if your analysis will have no point-by-point statistical uncertainty, the procedure will be more robust with the value of 0.01 as set above.

If you had some existing training points and wanted to keep those, while choosing additional points, you can uncomment the last two lines. The option `TPO_ReadPoints` will read the designated points in from file (the very place where you would write the new points) and use them as starting points. If the points are also designated by `TPO_FreezePoints`, then they will stay fixed throughout the search. For example, if you had points 0-9 already calculated and wanted to find the best positions for 10 more points, you would set `TPO_NMC`=20 and set both `TPO_ReadPoints` and `TPO_FreezePoints` to 0-9.

Now you can run `trainingpoint_optimizer`, which is meant to be run interactively. This will generate a list of coordinates in model-parameter space for each of $N_{\rm train}$ training points. Simply run the program by calling the command.

```
${MY_ANALYSIS}% ${SMOOTH_HOME}/bin/trainingpoint_optimizer
NTrainingpts=28, NMC=100000, LAMBDA=2.500000, ALPHA=0.010000
Optimize_MC: at beginning: bestSigma2=0.016793
+++ finished 1%, expected accuracy=0.0084093, success %=35.6, step size=0.0094491
+++ finished 2%, expected accuracy=0.007779, success %=11.6, step size=0.013928
 .
+++ finished 100%, expected accuracy=0.0072105, success %=23.1, step
↪  size=7.1374e-05
```

Your output might vary a bit because the random seed for the MCMC search is initialized by the current time. If you have defined your path to include `${SMOOTH_HOME}/bin/`, you can invoke the command by simply entering `trainingpoint_optimizer`.

The program writes information about the training points in the `smooth_data/FullModelRuns/` directory. Changing into that directory, there should now be 28 sub-directories, corresponding to the 28 training points: `FullModelRuns/run0`, `FullModelRuns/run1`, `FullModelRuns/run2` $\cdots$. Each directory has one text file describing the training points. For example, the `modelruns/run21/model_parameters.txt` file might be

```
par0 -39.9912
par1 -37.1959
par2 -106.002
par3 -47.4565
par4 -21.0126
par5 -83.7985
```

This describes the six model parameters, which will serve as the input for that model run. Given that the MCMC procedure was randomly seeded, the actual model parameters for the 21st point will vary. Even if one were to run with `TPO_NMC` set to infinity, and the points were in perfect position, the 28 points could be in different order.

In this example, `trainingpoint_optimizer` was run with `TPO_Method` set to `MCQuadratic`, which automatically set the number of training points to 28 given that there were six model parameters and 28 points are required to fully constrain a quadratic fit. If the option had been set to `MCSimplex`, 7 points would have been chosen, the minimum number to constrain a linear fit (If all the priors were Gaussian, this would result in a simplex arrangement). If `TPO_Method` had been set to `MC`, `trainingpoint_optimizer` would have used the option `TPO_NTrainingPts` to set the number of training points. The full range of options is described in Chapter 4.

10.5 Running the Full Model

Once the training points have been generated, you will need to run the full model at each training point. For the tutorial, a fake full model is provided. It reads the model-parameter values in each `smooth_data/modelruns/runI/model_parameters.txt` file and writes the corresponding observables for each point in model-parameter space, `I`, in the file `smooth_data/modelruns/runI/obs.txt`.

You need to run the program `fakefullmodel`, which is meant to be run interactively. Running the model,

```
${MY_ANALYSIS}% ${SMOOTH_HOME}/bin/fakefullmodel
NPars=6, NObs=10
Creating Fake Model Data for 28 Training Pts
```

The output simply shows the number of model parameters, observables, and training points. The number of model parameters was found by reading `smooth_data/Info/prior_info.tt` and the number of training points was determined by counting the number of files in `smooth_data/FullModelRuns`. The fake model is built on a Taylor expansion with random coefficients, chosen to be consistent with the form assumed by the emulator. This form is built on the choices of $\Lambda = 3.0$ and $\Sigma_A = 100$, so when the model is emulated the extracted hyper parameters can be compared to those values. Each of the 10 observables is assigned a separate randomly chosen set of coefficients.

Inspecting one of the output files, `smooth_data/FullModelRuns/run21/obs.txt`,

```
obs0 4.400155
obs1 -21.571182
obs2 77.834125
obs3 -90.449334
obs4 81.835445
obs5 109.049434
obs6 158.438724
obs7 35.845752
obs8 15.825137
obs9 117.629489
```

Your screen's output might be different if your random seeds are different. The second column provides the value of the specified observable for the full model at that specific training point. Additionally, `fakefullmodel` created a directory `smooth_data/FullModelTestingRuns/` which stores information about full-model runs at randomly chosen points in the model-parameter space. These points and their corresponding data will not be used for tuning the emulator. Instead, this data will be used below to test the emulator. You need not create such files for your real project, unless they wish to have some separate runs just for testing.

10.6 Training the Emulator and Testing it at the Training Points

Before building and tuning the emulator, you need to edit one additional file, the text file that sets numerous options for *Smooth Emulator*. This file is `smooth_data/Options/emulator_options.txt`. For the template used in this tutorial, the contents of that file appear as

```
# Location of output, comment out for interactive running
# LogFileName smoothlog.txt
SmoothEmulator_FixLambda false // If false will adjust optimize LAMBDA (false
 ↪  is default)
# Smoothness parameter to start maximizing procedure
SmoothEmulator_LAMBDA 2.5    // If SmoothEmulator_FixLambda is true, this fixes
 ↪  LAMBDA
#
# Location of training data, default is smooth_data/FullModelRuns
SmoothEmulator_FullModelRunsDirName smooth_data/FullModelRuns
#
# Choose which runs to use for training. Can be set to "all" or as list, e.g.
 ↪  1-5,8-12,15,18
SmoothEmulator_TrainingPts all
#
# Below only used for testing agains full model runs not used for training.
# Default location of testing data is smooth_data/FullModelTestingRuns/
SmoothEmulator_FullModelTestingRunsDirName smooth_data/FullModelTestingRuns
#
# List of points to be used for testing, all is default, can make list, e.g.
 ↪  1-5,8-12,15,18
SmoothEmulator_TestingPts all
#
# When calculating uncertainty, include(or not) the contribution from Lambda
 ↪  varying
SmoothEmulator_INCLUDE_LAMBDA_UNCERTAINTY true
#
```

More options are described in detail in Chapter 6.The options are mostly self-described by their names. To build an emulator the program needs to know the values of the observables as calculated by the full model at the training points. That is stored in the `smooth_data/FullModelRuns/` directory as described above. Files from a different directory can be used if the option

SmoothEmulator_FullModelRunsDirName is set to a different location. You can also choose which training points are used in the directory through the SmoothEmulator_TrainingPts option.

Here, you will run the program ${SMOOTH_HOME}/bin/smoothy_testattrainingpts. The corresponding source code for the first is
${SMOOTH_HOME}/software/MainPrograms/smoothy_testattrainingpts_main.cc,

```
int main(){
   NBandSmooth::CSmoothMaster master;
   master.TuneAllY();
   master.TestAtTrainingPts();
   return 0;
}
```

The options are set and the training data is read in by calling the constructor. The master object stores information for all the emulators, as there is one per each observable. The options training data are all read in by the constructor for the CSmoothMaster object when it is instantiated. As suggested by the name, TuneAllY() builds and trains emulators for all the observables. The method TestAtTrainingPts() prints out a comparison of the emulator values to those of the full model. Because the point-to-point noise (α) was set to zero in smooth_data/Info/observable_info.txt, the emulator should exactly reproduce the full-model calculations at each of the training points and for each of the observables. Thus, the emulator uncertainty at the training points should be nearly zero (would be zero if not for numerical accuracy).

Running this code:

```
${MY_ANALYSIS}% ${SMOOTH_HOME}/bin/smoothy_testattrainingpts
---------- obs0 -------------
---- Lambda=2.564895, SigmaA=68.194946
- itrain --- Y_full     Y_emulator    Sigma_emulator
 0 Y[0]=-6.596e+01 =? -6.596e+01  +/-  8.85541e-06
 1 Y[0]=-1.437e+01 =? -1.437e+01  +/-  7.29184e-06
 2 Y[0]=-7.470e+01 =? -7.470e+01  +/-  1.24567e-05
 .
 27 Y[0]= 1.958e+01 =?  1.958e+01  +/-  7.04033e-06
---------- obs1 -------------
---- Lambda=3.234328, SigmaA=114.745106
- itrain --- Y_full     Y_emulator    Sigma_emulator
 0 Y[1]= 3.557e+01 =?  3.557e+01  +/-  2.99640e-05
 1 Y[1]= 1.226e-01 =?  1.226e-01  +/-  1.70426e-05
  .
```

The second and third columns should be very close to one another, and the fourth column (the emulator uncertainty) should be close to zero. Indeed this is the case, though the numerical accuracy of the linear algebra routines does result in a small non-zero uncertainty. This thus represents a test of the procedure. Each emulator also optimizes a choice for the two hyper-parameters, Λ and σ_A. Given that the full model was created by a random Taylor expansion consistent with $\Lambda = 3$ and $\sigma_A = 100$, one would expect the system to choose them. But, given the very finite number of training points, there can be significant differences. By reading smoothdraft.pdf one can see that although there is often a significant deviation of a given observable's extracted hyper parameters, if one averages over many observables the expected value is rather well reproduced.

10.7 Testing the Emulator at Points Not Used for Training

In this case you will run the program `${SMOOTH_HOME}/software/bin/smoothy_testvsfullmodel`. In addition to building and training the emulator, exactly as was done above, this program reads information about runs not used in the testing. This was set in the options file by the `SmoothEmulator_FullModelTestingRunsDirName` and `SmoothEmulator_TestingPts` options. The source code for the main program, `${SMOOTH_HOME}/software/MainPrograms/smoothy_testvsfullmodel.cc`, is again rather simple.

```
int main(){
   NBandSmooth::CSmoothMaster master;
   master.TuneAllY();
   master.TestVsFullModel();
   return 0;
}
```

The data for the full model was created by the `fakefullmodel` program for 50 random points, and is found in smooth_data/FullModelTestingRuns. Running the program,

```
${MY_ANALYSIS}% smoothy_testvsfullmodel
iY=0: (<(Y-Yreal)^2>/SigmaY^2_emulator)^1/2=1.857423
percent < 1 sigma = 44.000000
iY=1: (<(Y-Yreal)^2>/SigmaY^2_emulator)^1/2=0.965030
percent < 1 sigma = 72.000000
iY=2: (<(Y-Yreal)^2>/SigmaY^2_emulator)^1/2=1.138670
percent < 1 sigma = 65.000000
iY=3: (<(Y-Yreal)^2>/SigmaY^2_emulator)^1/2=3.380905
percent < 1 sigma = 65.000000
iY=4: (<(Y-Yreal)^2>/SigmaY^2_emulator)^1/2=1.080057
percent < 1 sigma = 58.000000
iY=5: (<(Y-Yreal)^2>/SigmaY^2_emulator)^1/2=0.833119
percent < 1 sigma = 77.000000
iY=6: (<(Y-Yreal)^2>/SigmaY^2_emulator)^1/2=1.698096
percent < 1 sigma = 54.000000
iY=7: (<(Y-Yreal)^2>/SigmaY^2_emulator)^1/2=1.739706
percent < 1 sigma = 46.000000
iY=8: (<(Y-Yreal)^2>/SigmaY^2_emulator)^1/2=1.089194
percent < 1 sigma = 63.000000
iY=9: (<(Y-Yreal)^2>/SigmaY^2_emulator)^1/2=0.874994
percent < 1 sigma = 80.000000
```

Your results should look similar in principle, but will vary in detail because different random points were chosen. You can now peruse the files in `smooth_data/output_stuff/fullmodel_testdata/` and see the comparison of the 50 emulator values to the values calculated by `fakefullmodel`.

You can view this comparison by applying a Python plotting program, `${MY_ANALYSIS}/figs/YvsY/YvsY.py`. This program uses the Python package Matplotlib. Before running the plotting program, one must first create two information files. The

first is ${MY_ANALYSIS}/figs/figdata/prior_info.txt. It differs from the one in the smooth_data/Info/ directory in that it has only three columns, though the first columns are identical. You should first copy the files ${MY_ANALYSIS}/smooth_data/Info/prior_info.txt and ${MY_ANALYSIS}/smooth_data/Info/observable_info.txt to ${MY_ANALYSIS}/figs/figdata/.

```
..% cp ${MY_ANALYSIS}/smooth_data/Info/prior_info.txt ${MY_ANALYSIS}/figs/figdata/
..% cp ${MY_ANALYSIS}/smooth_data/Info/observable_info.txt
↪  ${MY_ANALYSIS}/figs/figdata/
```

Then edit the new prior_info.txt file. Keeping only the first two columns, add a third column which provides the LaTeX-like strings that Matplotlib would use to label the axes. The format for these strings differs from LaTeXstrings in some ways. For example \r must be \\r so as not to conflict with the basic file parsing. The new file can read:

```
  par0 gaussian $\theta_0$
  par1 uniform  $\theta_1$
  par2 gaussian $\theta_2$
  par3 uniform  $\theta_3$
  par4 gaussian $\theta_4$
  par5 uniform  $\theta_5$
```

Next, edit ${MY_ANALYSIS}/figs/figdata/observable_info.txt. In this case keep only the first column and list the LaTeX-like strings into the second column to be used to label axes in the plot. The new file can read:

```
obs0  $Y_0$
obs1  $Y_1$
obs2  $Y_2$
obs3  $Y_3$
obs4  $Y_4$
obs5  $Y_5$
obs6  $Y_6$
obs7  $Y_7$
obs8  $Y_8$
obs9  $Y_9$
```

The directory that stores the emulator-vs.-model comparison must be copied next. This directory contains information about the resulting observables for the full model evaluated at 100 randomly chosen points in model-parameter space, along with the emulator predictions. When fakefullmodel was run it created data for the full-model calculations and stored them in smooth_data/FullModelTestingRuns/. When smoothy_testvsfullmodel was run, it first read in the data for those points, then calculated the emulator predictions alongside the true full-model values. This comparison was written to separate files for each observable. These files are in the the directory smooth_data/output_stuff/fullmodel_testdata/.To copy the directory to a place where it can be read by the plotting program,

```
..% cp -r ${MY_ANALYSIS}/smooth_data/output_stuff/fullmodel_testdata
↪  ${MY_ANALYSIS}/figs/figdata/
```

The plotting script is now ready to be executed.

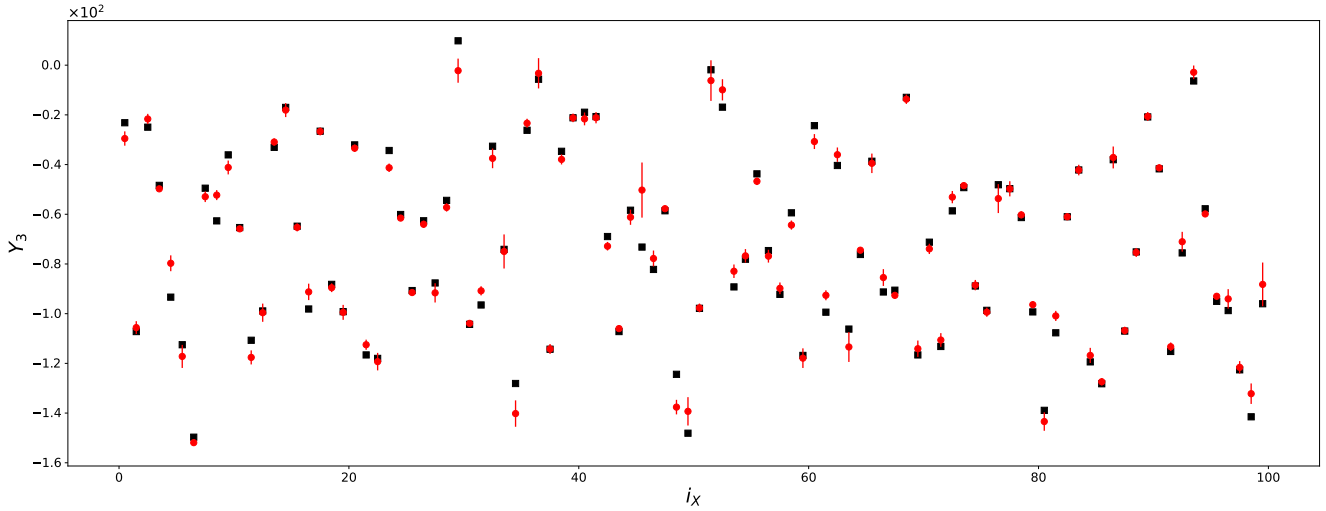


Figure 10.1: Comparison of full-model values (black squares) to emulator values (red circles) for 100 points in model-parameter space. The uncertainties are solely those associated with the emulation. If the uncertainties were accurately expressed, 68% of the points would lie within the uncertainty intervals, and in this case 65 emulator predictions were within the emulator's stated uncertainty. It is not unusual for the estimated uncertainty to be somewhat higher or lower what you would find, even if you were to evaluate a very large number of points.

```
..% cd ${MY_ANALYSIS}/figs/YvsY
${MY_ANALYSIS}/figs/figdata% python3 YvsY.py
--- Observables ---
iY= 0   obs0
iY= 1   obs1
iY= 2   obs2
iY= 3   obs3
iY= 4   obs4
iY= 5   obs5
iY= 6   obs6
iY= 7   obs7
iY= 8   obs8
iY= 9   obs9
Enter iY to designate observable: XXX
XXX of 100  points within 1 sigma
```

The script prompts you to identify which observable you wish to consider. For the 10 observables, enter the corresponding value of `iY`. The program will tell out of 100 points how many for how many the emulator prediction was within ± the stated uncertainty of the full-model value. If the uncertainty is accurately stated rougly 68% of the emulator predictions should fall within this range. A plot, `YvsY.pdf`, is created and presented in Fig. 10.1

The two programs demonstrated above, `smoothy_testattrainingpts` and `smoothy_testvsfullmodel`, both built and trained the emulator. If the emulator has already been built and trained from a previous run, one can save computational cost by using the hyper-parameters stored from a previous training. These are in

`smooth_data/output_stuff/sigmalambda.txt`. If you write a main program, you can then replace the `master.TuneAllY()` line with two lines: `master.ReadSigmaLambda()` followed by `master.TuneAllYFixedLambda`. By knowing and fixing Λ the tuning is very fast. Unless the model-parameter space is a high dimension, this is rarely worth the effort because tuning might take only a second or two. More detailed explanations can be found in Chapter 6.

10.8 Exploring and Analyzing the Posterior via MCMC

Given the experimental information, which is stored in `smooth_data/Info/experimental_info.txt`, one can then use the tuned emulator to explore the posterior likelihood through MCMC, which works via a Metropolis algorithm. The file `smooth_data/Info/experimental_info.txt` provided in the template is:

```
obs0  -89.734677  10.0 0.0
obs1  -21.483344  10.0 0.0
obs2  138.291439  10.0 0.0
obs3  -75.709000  10.0 0.0
obs4  78.120413  10.0 0.0
obs5  55.999272  10.0 0.0
obs6  134.152043  10.0 0.0
obs7  140.861162  10.0 0.0
obs8  8.300424  10.0 0.0
obs9  105.730671  10.0 0.0
```

The first column is the list of observable names, which should be identical to those listed in `smooth_data/Info/observable_info.txt`. The second and third columns lists the experimental measurement, Y_a, and the experimental uncertainty, $\sigma_a^{\rm exp}$. The last column lists the additional uncertainty due to shortcomings of the model, $\sigma_a^{\rm theory}$, i.e. missing or not exactly represented physics. For the purposes of comparing theory to data, only the combination $(\sigma_a^{\rm exp})^2 + (\sigma_a^{\rm theory})^2 + (\sigma_a^{\rm emu})^2$ comes into play, because this combination appears in the likelihood for the posterior,

$$\mathcal{L}(\vec{\theta}) = \prod_a \frac{1}{\sqrt{2\pi(\sigma_a^{\rm tot})^2}} \exp\left\{-\frac{(Y_a(\vec{\theta}) - Y_a^{\rm exp})^2}{2(\sigma_a^{\rm tot})^2}\right\} \tag{10.1}$$
$$(\sigma_a^{\rm tot})^2 = (\sigma_a^{\rm exp})^2 + (\sigma_a^{\rm theory})^2 + (\sigma_a^{\rm emu})^2.$$

Whereas the emulator uncertainty, $\sigma_a^{\rm emu}$, depends on the location in parameter space, $\vec{\theta}$, the other two contributions are assumed to be independent of $\vec{\theta}$.

There are options for the MCMC, which are stored in `smooth_data/Options/mcmc_options.txt`. For the tutorial template, that file is

```
# This is for the MCMC search of parameter space
# (not for the emulator tuning)
#LogFileName smoothlog.txt # comment out for interactive running
MCMC_METROPOLIS_STEPSIZE 0.06
MCMC_IGNORE_EMULATOR_ERROR false
MCMC_NBURN  100
```

```
MCMC_NTRACE 1000
MCMC_NSKIP  5
MCMC_IGNORE_EMULATOR_ERROR false
RANDY_SEED  12345
```

The Metropolis step size should be adjusted so that the Metropolis success rate is approximately one half. The success rate prints out when the `mcmc` code runs. If the success rate is anywhere between 20 and 80%, this should be fine. But, if the rate is close to zero or close to 100%, the efficiency of the procedure suffers. It is recommended to run the MCMC code with a modest number of steps, then adjust the step size accordingly. The parameter `MCMC_NBURN` sets the number of Metropolis steps to be used in the "burn-in" stage, i.e. before one begins to store the trace. The number of elements to store in the trace in `MCMC_NTRACE`, and `MCMC_NSKIP` sets the number of steps to skip before storing a new point in the trace. Thus, if `MCMC_NTRACE` is one million and if `MCMC_NSKIP`=5, then the procedure will perform 5 million steps, and store every fifth one, leading to one million stored points in the trace. Finally, `RANDY_SEED` sets the random number seed. Options are explained in greater detail in Chapter 7.

The source code, {`SMOOTH_HOME`}`/bin/smoothy_mcmc_main.cc`, is rather simple.

```
int main(){
   NBandSmooth::CSmoothMaster master;
   master.TuneAllY();
   //Next two lines can take replace of TuneAllY() if you have previously tuned
    ↪  and wish to speed calculations
   //master.ReadSigmaLambda(); //Next two lines can take place of TuneAllY() if
    ↪  you have previously tuned and wish to speed calculations
   //master.TuneAllYFixedLambda();
   NBandSmooth::CMCMC mcmc(&master);

   unsigned int Nburn=master.parmap->getI("MCMC_NBURN",1000);  // Steps for burn
    ↪  in
   unsigned int Ntrace=master.parmap->getI("MCMC_NTRACE",1000); // Record this
    ↪  many points
   unsigned int Nskip=master.parmap->getI("MCMC_NSKIP",5); // Only record every
    ↪  Nskip^th point

   mcmc.PerformTrace(1,Nburn);
   CLog::Info("finished burn in\n");

   mcmc.PruneTrace(); // Throws away all but last point
   mcmc.PerformTrace(Ntrace,Nskip);
   mcmc.WriteTrace(); // Writes trace
   mcmc.EvaluateTrace();

return 0;
}
```

After tuning the emulator the program creates an `CMCMC` object. It then calculates a trace for `Nburn` steps. During the burn-in stage none of the points are stored. The trace is then pruned, keeping only

the last point. Starting a new trace every `Nskip`$^{\text{th}}$ point is recorded, until the list of newly recorded points is `Ntrace` long. The trace is then written to file. Finally, the method `EvaluateTrace()` analyzes the trace and calculates several averages and covariances, which are then stored in the `smooth_data/MCMC/` directory in several files.

Running the MCMC program gives the following output:

```
${MY_ANALYSIS}/rhic% ${SMOOTH_HOME}/bin/smoothy_mcmc
At beginning of Trace, LL=-35.601204
At end of trace, best LL=-23.345938
Best Theta=
0.141293  0.154471  0.199519  0.254597  0.193075  0.203377
Metropolis success percentage=48.700000
finished burn in
Nburn=10000, Nskip=5, Ntrace=100000
At beginning of Trace, LL=-24.773906
finished 10%
finished 20%
finished 30%
finished 40%
finished 50%
finished 60%
finished 70%
finished 80%
finished 90%
finished 100%
At end of trace, best LL=-23.274102
Best Theta=
0.179306  0.190535  0.178967  0.214708  0.203395  0.186171
Metropolis success percentage=48.211200
writing trace, ntrace = 100001
```

Here `best LL` refers to the log-likelihood and `Best Theta` refers to the value of $\vec{\theta}$ that gave the maximum log-likelihood. Values of `Best Theta` and `best LL` are given after the burn-in and after the trace. The "experimental" values were generated by running the full model with all 6 values, θ_i, set to 0.2, so the fact that the best value of $\vec{\theta}$ found was close to that point is a validation of the method.

The trace is written to `smooth_data/MCMC_trace/trace_theta.txt`. It lists the scaled model parameters for each point in the trace,

```
theta_1 theta_2 theta_3 theta_4 ...
theta_1 theta_2 theta_3 theta_4 ...
theta_1 theta_2 theta_3 theta_4 ...
```

A similar file, `smooth_data/MCMC_trace/trace_X.txt`, shows the trace in terms of the unscaled model parameters. If `MCMC_NTRACE` is set to a million, there would be a million lines in each file.

To view the posterior likelihood, first move the trace information to a location where it can be read by the plotting program.

```
${MY_ANALYSIS}/rhic% cp ${MY_ANALYSIS}/smooth_data/MCMC/trace_theta.txt
↪  ${MY_ANALYSIS}/figs/figdata/
```

Next, move into the `figs/posterior/` directory and run the Python script:

```
../% cd ${MY_ANALYSIS}/figs/posterior
${MY_ANALYSIS}/figs/posterior% python3 posterior.py
${MY_ANALYSIS}/figs/posterior% Number of points in trace = 100001
```

The script should create a figure, `posterior.pdf`, shown in Fig. 10.2 The likelihood is projected for

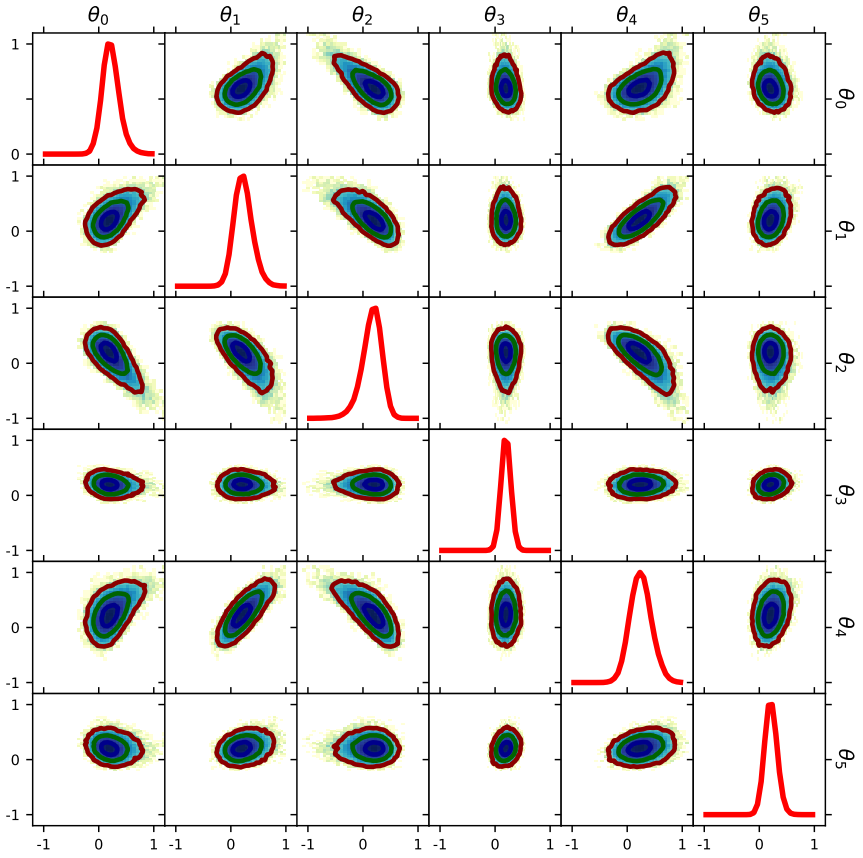


Figure 10.2: Projections for the posterior likelihood from the MCMC trace. The contour lines represent $1-\sigma$, $2-\sigma$ and $3-\sigma$ likelihoods.

individual model parameters (along the diagonal), or for pairs (off-diagonal). The plot is in terms of the scaled variables, θ_i. To translate to the true model-parameter ranges, one can look at the `smooth_data/Info/modelpars_info.txt` file, which gives the prior ranges of the model parameters before they are scaled to the -1 to 1 range. The file `figs/directions.pdf` shows how you can alter the plot. For example there is a line in the Python script, `ParsToPlot=[1,2,3,4,5]`, which you can edit to change the ordering of the model-parameters, and to choose which model parameters are considered.

10.8.1 Viewing the Resolving Power

The program also calculates various covariances: $\langle\langle \delta\theta_i \delta\theta_j \rangle\rangle$, $\langle\langle \delta\theta_i \delta Y_a \rangle\rangle$, and $\langle\langle \delta Y_a \delta Y_b \rangle\rangle$. The quantities $\langle\langle ....\rangle\rangle$ refer to averages over the posterior, i.e. averages over the trace. The eigenvalues and eigenvectors of $\langle\langle \delta\theta_i \delta\theta_j \rangle\rangle$ are also recorded. Hopefully, you will find the file names in `smooth_data/MCMC/` to be self-explanatory. The files describing the posterior-weighted average, $\langle\langle \vec{\theta} \rangle\rangle$, and the posterior covariances, $\langle\langle \delta\theta_i \delta\theta_j \rangle\rangle$, are also in the directory. Other file lists the eigenvectors and eigenvalues of the covariance matrix.

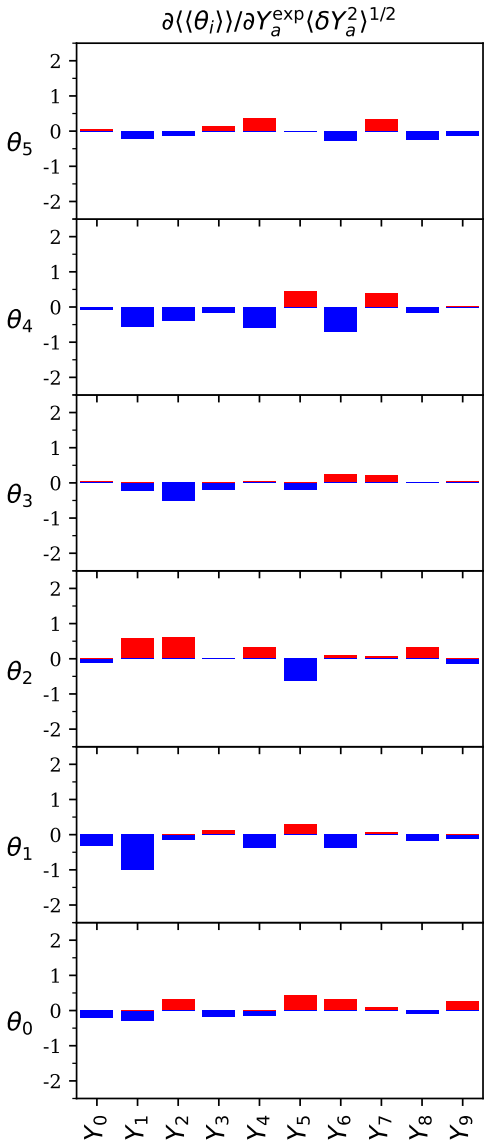


Figure 10.3: Resolving Power. Red bars represent positive correlations with $Y_a^{\rm exp}$ and θ_i. Larger bars suggest that the particular observable contributes more to the constraint of the particular model parameter.

One measure of the resolving power is:

$$\mathcal{R}_{ia} \equiv \frac{\partial\langle\langle\theta_i\rangle\rangle}{\partial Y_a^{\rm exp}} \langle \delta Y_a^2 \rangle^{1/2}. \tag{10.2}$$

This quantifies how the posterior value of θ_i changes as the experimental value changes if the experimental value, $Y_a^{\rm exp}$, changes an amount characteristic of the variance of Y_a across the

prior. Higher values of $\mathcal{R}_{ia}$ for different a demonstrate the relative contributions of different observables a to constrain a model parameter i. The resolving power matrix is written to `MCMC_trace/ResolvingPower.txt`. The resolving power is a function of the covariances listed above (see preprint.pdf). The data for the resolving power was also created by running `smoothy_mcmc`. To plot the resolving power move the data file `smooth_data/MCMC/ResolvingPower.txt` to `figs/figdata/`. You can then run the Python script:

```
..% cp ${MY_ANALYSIS}/smooth_data/MCMC/ResolvingPower.txt
↪  ${MY_ANALYSIS}/figs/figdata/
..% cd ${MY_ANALYSIS}/figs/resolvingpower
${MY_ANALYSIS}/figs/resolvingpower% python3 RP.py
NPars= 6  NObs= 10
```

The resulting figure, `RP.pdf`, is shown if Fig. 10.3

If you have an Apple computer, or if you are running Linux with the PDF viewer, `okular`, you can edit the Python scripts mentioned above and uncomment the approriate line shown in green and comment out the `plt.show()` and `plt.close()` lines. The lines at the end of `RP.py` are

```
 .
plt.show()
plt.close()
# if you have Mac OS and want to see pdf file, comment out previous two lines and
↪  uncomment line below
#os.system("open -a Preview "+outputfilename);
# if you have Linux and want to see pdf file, comment out previous two lines and
↪  uncomment line below
#os.system("okular "+outputfilename+"&");
 .
```

You can then see the actual PDF file while the script is running. Similar changes can be made to all three Python plotting scripts mentioned here.

11 Binding *Smooth Emulator* to Python Code

11.1 Overview

The User might wish to call the emulator functionality from within a Python script or from within a Python notebook. The ground work, based on pybind11, exists for calling some of the emulator functionality within a Python script. The User may create an object from within Python from which they can create and tune an emulator. Simple calls exist which permit the User to get emulator predictions and uncertainties as a function of either the model parameters, $\vec{X}$, or the scaled parameter, $\vec{\theta}$. Functions that translate back and forth between $\vec{X}$ and $\vec{\theta}$ are also provided.

The training-point-optimization and MCMC functionality is not available in the current software. However, this may be sufficient if a User wishes to build and tune an emulator using *Smooth Emulator* software, then use their own MCMC python-based software to explore the model-parameter space.

11.2 Setting Up The Software

The binding of C++ libraries to Python is accomplished through pybind11. The pybind11 functionality can often be installed through the `pip` Python package manager, but the User must be very careful that the installation is consistent with Python version being used. On a Mac, it was found that the best way to install pybind11 and python was to install all three through the system package manager, e.g. *homebrew*. Further, troubles on the Mac were found by using llvm/clang++, but the pybind11 software ran successfully using g++. If the User has *homebrew* installed, all the Python functionality can be installed with the commands:

```
% brew install python
% brew install numpy
% brew install python-matplotlib
% brew install pybind11
```

The User should now compile the software:

```
..% cd ${SMOOTH_HOME}/software/pybind_stuff
${SMOOTH_HOME}/software/pybind_stuff% cmake .
${SMOOTH_HOME}/software/pybind_stuff% make
```

11.3 Editing and Running a Python Script

The Python script must be located in a analysis directory, i.e. one where there is a `smooth_data/` directory set up in the manner required by the emulator software. An example Python script is in the `${SMOOTH_HOME}/software/pybind_stuff/` directory. It can be moved to the analysis directory.

```
..% cp ${SMOOTH_HOME}/software/pybind_stuff/smoothy_emulate.py ${MY_ANALYSIS}/
..% cd ${MY_ANALYSIS}/
```

If the User has defined an environment variable, ${SMOOTH_HOME}, to the parent directory of software/pybind_stuff, the script is ready to run. Otherwise the User needs to edit the following line from the Python script.

```
sys.path.insert(0,os.environ['SMOOTH_HOME']+"/software/pybind_stuff")
```

The User can switch out this line for

```
sys.path.insert(0,os."XXXXXX/software/pybind_stuff")
```

Here XXXXXX is replaced by the absolute path.

To run the script,

```
${MY_ANALYSIS}% python3 ${SMOOTH_HOME}/software/pybind_stuff/smoothy_emulate.py
Adding this to path: XXXXXX
X= [20. 20. 20. 20. 20. 20.]
Y[ 0 ]= 62.06376520471999  SigmaY= 0.46481585069318604
Y[ 1 ]= -88.61735278099212  SigmaY= 2.198210063746222
Y[ 2 ]= 35.35391376800502  SigmaY= 0.5431950258044482
Y[ 3 ]= 47.38398026724699  SigmaY= 0.5632478557065526
Y[ 4 ]= 41.16532652137935  SigmaY= 0.29387511492428314
Y[ 5 ]= 64.91706104229243  SigmaY= 0.23423530131223003
Y[ 6 ]= -192.62294658870263  SigmaY= 0.04820162692778329
Y[ 7 ]= 88.43393115018455  SigmaY= 0.3480224686688987
Y[ 8 ]= 3.840354622268825  SigmaY= 0.5590394534630352
Y[ 9 ]= 64.70923479158401  SigmaY= 0.7742206531639135
```

The script took the point in model-parameter space, $(20, 20, 20, 20, 20, 20)$, and calculated all three observables along with their uncertainties.

The user might find it easier to move the Python script to the current directory so that they can avoid typing in the long path. The Python script, smoothy_emulate.py, is

```
import numpy as np
import sys
import os
print('Adding this to path: '+os.environ['SMOOTH_HOME'])
sys.path.insert(0,os.environ['SMOOTH_HOME']+"/software/pybind_stuff")
import emulator_smooth
smoothmaster=emulator_smooth.emulator_smooth()
smoothmaster.TuneAllY()
NPars=smoothmaster.GetNPars()
NObs=smoothmaster.GetNObs()

X=np.zeros(NPars,dtype='float')
Y=np.zeros(NObs,dtype='float')
SigmaY=np.zeros(NObs,dtype='float')

X[0]=20
```

```
X[1]=20
X[2]=20
X[3]=20
X[4]=20
X[5]=20
print('X=',X)

for iY in range(0,NObs):
  Y[iY],SigmaY[iY]=smoothmaster.GetYSigmaFromXPython(iY,X)
  print('Y[',iY,']=',Y[iY],' SigmaY=',SigmaY[iY])
```

There are other calls available with the current software. One could add some version of the lines below to their Python script.

```
theta=smoothmaster.GetThetaFromX(X)
X=smoothmaster.GetXFromTheta(theta)
Y[iY],SigmaY[iY]=smoothmaster.GetYSigmaFromThetaPython(iY,theta)
Y[iY]=smoothmaster.GetYOnlyFromXPython(iY,X)
Y[iY]=smoothmaster.GetYOnlyFromThetaPython(iY,theta)
```

Hopefully, the User will find the functionality of this code to be self-explanatory. These calls should be sufficient to incorporate training and calling the emulator into any Python script.

11.4 Making More Functionality Available via Python

The easiest way to add more functions is to edit and extend the file `${SMOOTH_HOME}/software/pybind_stuff/pybind_main.cc`. One can see how all the functions above were defined. One can extend this to more of the functions defined in the header file `${SMOOTH_HOME}/software/include/msu_smooth/master.h`. However, one should remember that the binding cannot easily change the functionality of functions with arguments passed by reference. Thus, if the User wishes to find some variable, e.g. `Z`, from some C++ function that is written in the form `void ThisIsMyMethodToGetZ(double &Z,double x,double y)`, they will instead have to write a new C++ function in the form `double ThisIsMyMethodToGetZ(double x,double z)`, where the new function returns `Z` rather than having it updated by the function. For example to return the emulated value and uncertainty, a function was created of the form `vector<double> GetYSigmaFromXPython(int iY,vector<double> X)`. The returned vector was 2-dimensional with `Y[iY]` being the first element and `sigmaY[iY]` being the second element.